

Практична робота

(Модульний контроль №2, Частина 1)

**З дисципліни «Інформаційно-технічний
супровід програмного забезпечення»**



Виконав:

Студент 3 курсу

3 групи ІПЗ

Сідун Владислав Владиславович

Тема: Формування пакета супровідної документації

Мета: Сформувати пакет документів для розробленого ПЗ.

Виконання

Варіант 11: Інтелектуальна система оцінювання ризику проекту.

ІНСТРУКЦІЯ КОРИСТУВАЧА

Project Risk AI v1.0.0

Інтелектуальна система оцінювання ризику проекту

1. Призначення програмного забезпечення

Project Risk AI — це програмна система призначена для автоматизованого оцінювання ризику зриву термінів ІТ-проектів на основі алгоритмів машинного навчання. Система дозволяє керівникам проектів та аналітикам швидко отримати оцінку ризику на основі ключових параметрів проекту без необхідності глибоких технічних знань у сфері машинного навчання.

Основні функції системи:

- Введення та валідація параметрів проекту через зручний графічний інтерфейс
- Автоматичне навчання ML-моделі на синтетичних даних
- Визначення рівня ризику (Низький / Середній / Високий) з відображенням ймовірностей
- Візуалізація результатів у вигляді діаграм
- Генерація структурованого звіту у форматі Excel
- Налаштування параметрів програми через зовнішній конфігураційний файл
- Автоматичне логування всіх дій та помилок

2. Технічні вимоги

Апаратні вимоги:

- Процесор: Intel Core i3 або аналог (тактова частота від 1.6 ГГц)
- Оперативна пам'ять: не менше 4 ГБ RAM
- Вільний простір на диску: не менше 500 МБ
- Монітор з роздільністю не менше 1280×720 пікселів

Програмні вимоги:

- Операційна система: Windows 10/11, macOS 12+, Ubuntu 20.04+
- Python версії 3.11 або вище
- Git (для клонування репозиторію)

3. Встановлення та налаштування

Крок 1 — Клонування репозиторію

Відкрийте термінал та виконайте команду:

```
git clone https://github.com/VladSSidun/Info_Tech_Support_Modul1.git  
cd Info_Tech_Support_Modul1
```

Крок 2 — Створення віртуального середовища

```
python -m venv venv
```

Активація на Windows (Git Bash): `source venv/Scripts/activate`

Активація на macOS/Linux: `source venv/bin/activate`

Після активації у терміналі з'явиться префікс (venv).

Крок 3 — Встановлення залежностей

```
pip install -r requirements.txt
```

```

vlad0@Vlad_legion MINGW64 ~/OneDrive/Робочий стіл/Освіта/UNIVERSITY/6 семестр/Інфо-тех-Супровід/example (master)
● $ git clone https://github.com/VladSSidun/Info_Tech_Support_Modul1
Cloning into 'Info_Tech_Support_Modul1'...
remote: Enumerating objects: 50, done.
remote: Counting objects: 100% (50/50), done.
remote: Compressing objects: 100% (34/34), done.
remote: Total 50 (delta 14), reused 50 (delta 14), pack-reused 0 (from 0)
Receiving objects: 100% (50/50), 28.61 KiB | 3.18 MiB/s, done.
Resolving deltas: 100% (14/14), done.

vlad0@Vlad_legion MINGW64 ~/OneDrive/Робочий стіл/Освіта/UNIVERSITY/6 семестр/Інфо-тех-Супровід/example (master)
● $ cd Info_Tech_Support_Modul1

vlad0@Vlad_legion MINGW64 ~/OneDrive/Робочий стіл/Освіта/UNIVERSITY/6 семестр/Інфо-тех-Супровід/example/Info_Tech_Support_Modul1 (task1)
$ python -m venv venv
● source venv/Scripts/activate
(venv)

```

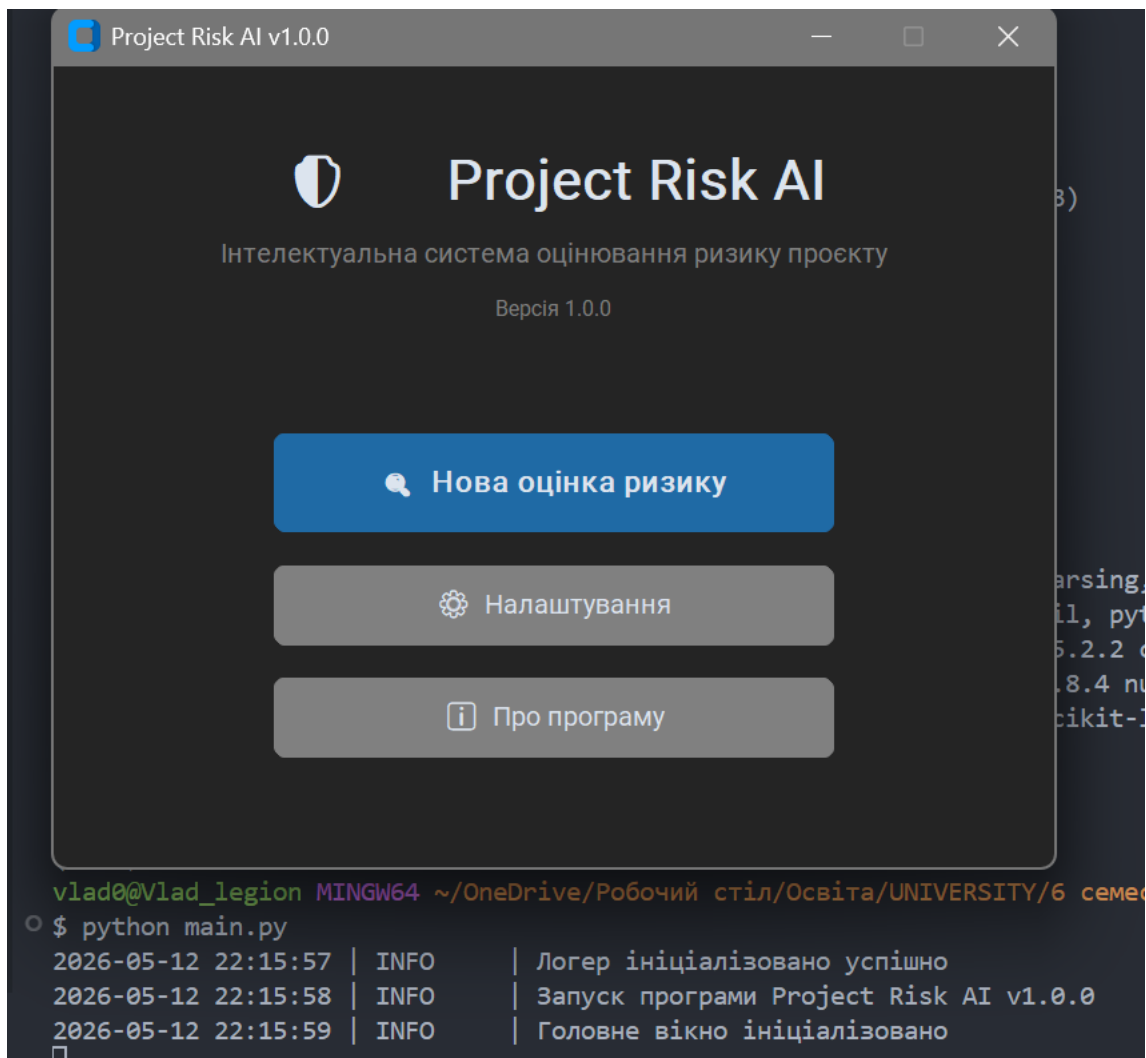
```

(venv)
vlad0@Vlad_legion MINGW64 ~/OneDrive/Робочий стіл/Освіта/UNIVERSITY/6 семестр/Інфо-тех-Супровід/example/Info_Tech_Support_Modul1 (task1)
○ $ pip install -r requirements.txt
Collecting customtkinter==5.2.2 (from -r requirements.txt (line 1))
  Using cached customtkinter-5.2.2-py3-none-any.whl.metadata (677 bytes)
Collecting scikit-learn==1.4.2 (from -r requirements.txt (line 2))
  Using cached scikit_learn-1.4.2-cp311-cp311-win_amd64.whl.metadata (11 kB)
Collecting pandas==2.2.2 (from -r requirements.txt (line 3))
  Using cached pandas-2.2.2-cp311-cp311-win_amd64.whl.metadata (19 kB)
Collecting numpy==1.26.4 (from -r requirements.txt (line 4))
  Using cached numpy-1.26.4-cp311-cp311-win_amd64.whl.metadata (61 kB)
Collecting matplotlib==3.8.4 (from -r requirements.txt (line 5))
  Using cached matplotlib-3.8.4-cp311-cp311-win_amd64.whl.metadata (5.9 kB)
Collecting joblib==1.4.2 (from -r requirements.txt (line 6))
  Using cached joblib-1.4.2-py3-none-any.whl.metadata (5.4 kB)
Collecting fpdf2==2.7.9 (from -r requirements.txt (line 7))
  Using cached fpdf2-2.7.9-py2.py3-none-any.whl.metadata (58 kB)
Collecting openpyxl==3.1.2 (from -r requirements.txt (line 8))
  Using cached openpyxl-3.1.2-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting pytest==8.2.0 (from -r requirements.txt (line 9))
  Using cached pytest-8.2.0-py3-none-any.whl.metadata (7.5 kB)
Collecting darkdetect (from customtkinter==5.2.2->-r requirements.txt (line 1))
  Using cached darkdetect-0.8.0-py3-none-any.whl.metadata (3.6 kB)
Collecting packaging (from customtkinter==5.2.2->-r requirements.txt (line 1))
  Using cached packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
Collecting scipy>=1.6.0 (from scikit-learn==1.4.2->-r requirements.txt (line 2))
  Using cached scipy-1.17.1-cp311-cp311-win_amd64.whl.metadata (60 kB)
Collecting threadpoolctl>=2.0.0 (from scikit-learn==1.4.2->-r requirements.txt (line 2))
  Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting python-dateutil>=2.8.2 (from pandas==2.2.2->-r requirements.txt (line 3))
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas==2.2.2->-r requirements.txt (line 3))
  Using cached pytz-2026.2-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas==2.2.2->-r requirements.txt (line 3))
  Using cached tzdata-2026.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting contourpy>=1.0.1 (from matplotlib==3.8.4->-r requirements.txt (line 5))
  Using cached contourpy-1.3.3-cp311-cp311-win_amd64.whl.metadata (5.5 kB)
Collecting cycycler>=0.10 (from matplotlib==3.8.4->-r requirements.txt (line 5))
  Using cached cycycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib==3.8.4->-r requirements.txt (line 5))
  Using cached fonttools-4.62.1-cp311-cp311-win_amd64.whl.metadata (119 kB)
Collecting kiwisolver==1.3.1 (from matplotlib==3.8.4->-r requirements.txt (line 5))
  Using cached kiwisolver-1.5.0-cp311-cp311-win_amd64.whl.metadata (5.2 kB)
Collecting pillow>=8 (from matplotlib==3.8.4->-r requirements.txt (line 5))
  Using cached pillow-12.2.0-cp311-cp311-win_amd64.whl.metadata (9.0 kB)
Collecting pyparsing>=2.3.1 (from matplotlib==3.8.4->-r requirements.txt (line 5))
  Using cached pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Collecting defusedxml (from fpdf2==2.7.9->-r requirements.txt (line 7))
  Using cached defusedxml-0.7.1-py2.py3-none-any.whl.metadata (32 kB)
Collecting et_xmlfile (from openpyxl==3.1.2->-r requirements.txt (line 8))
  Using cached et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Collecting iniconfig (from pytest==8.2.0->-r requirements.txt (line 9))
  Using cached iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)

```

```
Collecting iniconfig (from pytest==8.2.0->-r requirements.txt (line 9))
  Using cached iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Collecting pluggy<2.0,>=1.5 (from pytest==8.2.0->-r requirements.txt (line 9))
  Using cached pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting colorama (from pytest==8.2.0->-r requirements.txt (line 9))
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas==2.2.2->-r requirements.txt (line 3))
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Using cached customtkinter-5.2.2-py3-none-any.whl (296 kB)
Using cached scikit_learn-1.4.2-cp311-cp311-win_amd64.whl (10.6 MB)
Using cached pandas-2.2.2-cp311-cp311-win_amd64.whl (11.6 MB)
Using cached numpy-1.26.4-cp311-cp311-win_amd64.whl (15.8 MB)
Using cached matplotlib-3.8.4-cp311-cp311-win_amd64.whl (7.7 MB)
Using cached joblib-1.4.2-py3-none-any.whl (301 kB)
Using cached fpdf2-2.7.9-py2.py3-none-any.whl (206 kB)
Using cached openpyxl-3.1.2-py2.py3-none-any.whl (249 kB)
Using cached pytest-8.2.0-py3-none-any.whl (339 kB)
Using cached contourpy-1.3.3-cp311-cp311-win_amd64.whl (225 kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Using cached fonttools-4.62.1-cp311-cp311-win_amd64.whl (2.3 MB)
Using cached kiwisolver-1.5.0-cp311-cp311-win_amd64.whl (73 kB)
Using cached packaging-26.2-py3-none-any.whl (100 kB)
Using cached pillow-12.2.0-cp311-cp311-win_amd64.whl (7.1 MB)
Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
Using cached pyparsing-3.3.2-py3-none-any.whl (122 kB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached pytz-2026.2-py2.py3-none-any.whl (510 kB)
Using cached scipy-1.17.1-cp311-cp311-win_amd64.whl (36.6 MB)
Using cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Using cached tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Using cached darkdetect-0.8.0-py3-none-any.whl (9.0 kB)
Using cached defusedxml-0.7.1-py2.py3-none-any.whl (25 kB)
Using cached et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: pytz, tzdata, threadpoolctl, six, pyparsing, pluggy, pillow, packaging, numpy, kiwisolver, joblib, iniconfig, fonttools, et-xml
file, defusedxml, darkdetect, cycler, colorama, scipy, python-dateutil, pytest, openpyxl, fpdf2, customtkinter, contourpy, scikit-learn, pandas, matplotlib
Successfully installed colorama-0.4.6 contourpy-1.3.3 customtkinter-5.2.2 cycler-0.12.1 darkdetect-0.8.0 defusedxml-0.7.1 et-xmlfile-2.0.0 fonttools-4.62.1 fp
df2-2.7.9 iniconfig-2.3.0 joblib-1.4.2 kiwisolver-1.5.0 matplotlib-3.8.4 numpy-1.26.4 openpyxl-3.1.2 packaging-26.2 pandas-2.2.2 pillow-12.2.0 pluggy-1.6.0 py
parsing-3.3.2 pytest-8.2.0 python-dateutil-2.9.0.post0 pytz-2026.2 scikit-learn-1.4.2 scipy-1.17.1 six-1.17.0 threadpoolctl-3.6.0 tzdata-2026.2

[notice] A new release of pip is available: 24.0 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```



Крок 4 — Перевірка конфігурації

Відкрийте файл `config.json` у текстовому редакторі. Переконайтесь що він містить коректні налаштування:

```
{  
  "app": {  
    "name": "Project Risk AI",  
    "version": "1.0.0"  
  },  
  "appearance": {  
    "theme": "dark",  
    "color_scheme": "blue"  
  }  
}
```

```
},  
"paths": {  
  "log_file": "logs/app.log",  
  "model_file": "data/model.pkl",  
  "reports_dir": "data/reports"  
}  
}
```

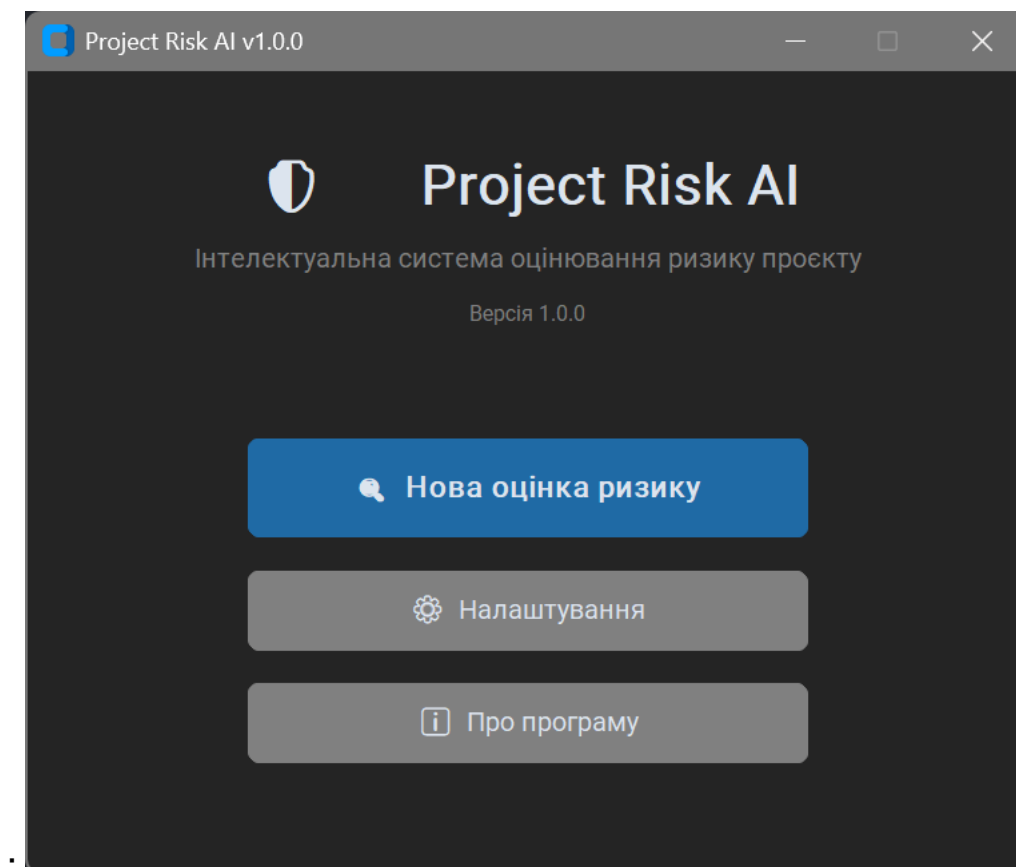
```
{  
  "app": {  
    "name": "Project Risk AI",  
    "version": "1.0.0",  
    "language": "uk"  
  },  
  "appearance": {  
    "theme": "dark",  
    "color_scheme": "blue"  
  },  
  "paths": {  
    "log_file": "logs/app.log",  
    "model_file": "data/model.pkl",  
    "data_file": "data/sample_data.csv",  
    "reports_dir": "data/reports"  
  },  
  "model": {  
    "n_estimators": 100,  
    "random_state": 42,  
    "test_size": 0.2  
  }  
}
```

4. Запуск програми

Переконайтесь що віртуальне середовище активоване (є префікс (venv)), після чого виконайте:

```
python main.py
```

```
2026-05-12 21:25:48 | INFO | Логер ініціалізовано успішно
2026-05-12 21:25:49 | INFO | Запуск програми Project Risk AI v1.0.0
2026-05-12 21:25:49 | INFO | Головне вікно ініціалізовано
```

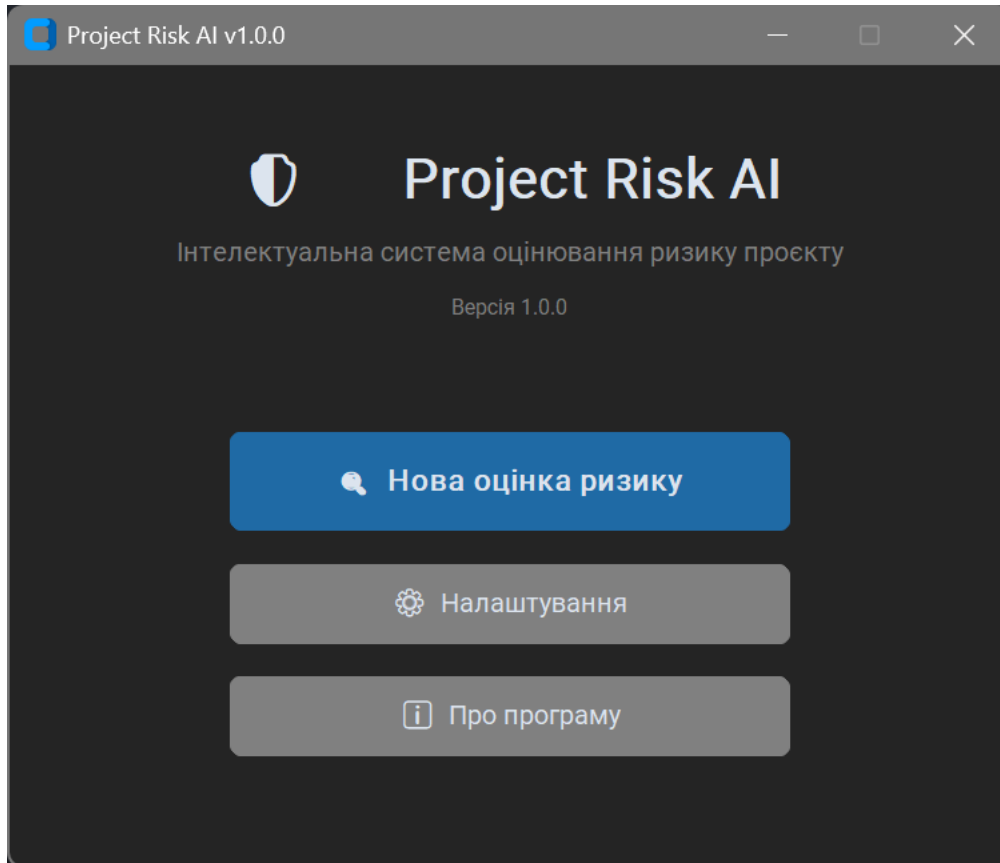


5. Опис інтерфейсу користувача

Програма має чотири вікна: головне вікно, форма введення параметрів, вікно результатів та вікно налаштувань.

5.1 Головне вікно

Головне вікно є стартовою точкою програми. Воно містить назву системи, версію та три кнопки навігації.



Елементи головного вікна:

Кнопка "Нова оцінка ризику" — відкриває форму введення параметрів проєкту для проведення нової оцінки. Це основна функція програми.

Кнопка "Налаштування" — відкриває вікно налаштувань де можна змінити тему інтерфейсу та параметри моделі.

Кнопка "Про програму" — відображає діалогове вікно з інформацією про назву системи, версію та використаний алгоритм.

5.2 Форма введення параметрів

Форма містить вісім полів для введення характеристик проєкту. Кожне поле має назву параметра, підказку з допустимим діапазоном значень та поле для введення.

Параметри проекту

Заповніть всі поля для отримання оцінки ризику

Кількість учасників команди

Введіть кількість людей у команді (1–50)

Досвід команди (роки)

Середній досвід учасників у роках (0–20)

Бюджет проекту (тис. грн)

Загальний бюджет проекту в тисячах гривень (10–10000)

Тривалість проекту (тижні)

Планована тривалість проекту в тижнях (1–104)

Кількість вимог

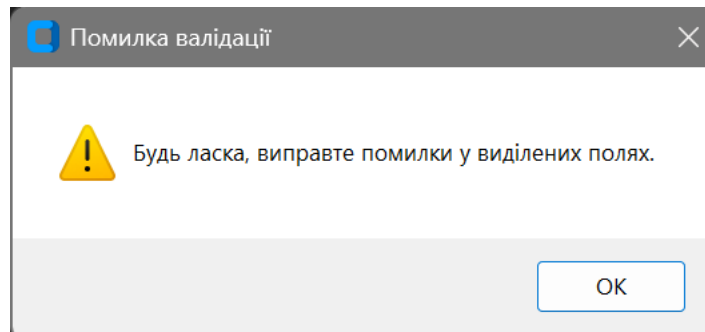
Загальна кількість функціональних вимог (1–500)

Чіткість вимог (0–1)

Наскільки чіткі вимоги: 0 = повна невизначеність, 1 = повна ясність

🗨 Оцінити ризик

🔄 Очистити



Кількість учасників команди

Введіть кількість людей у команді (1 – 50)

⚠ Введіть ціле число

Параметри що вводяться:

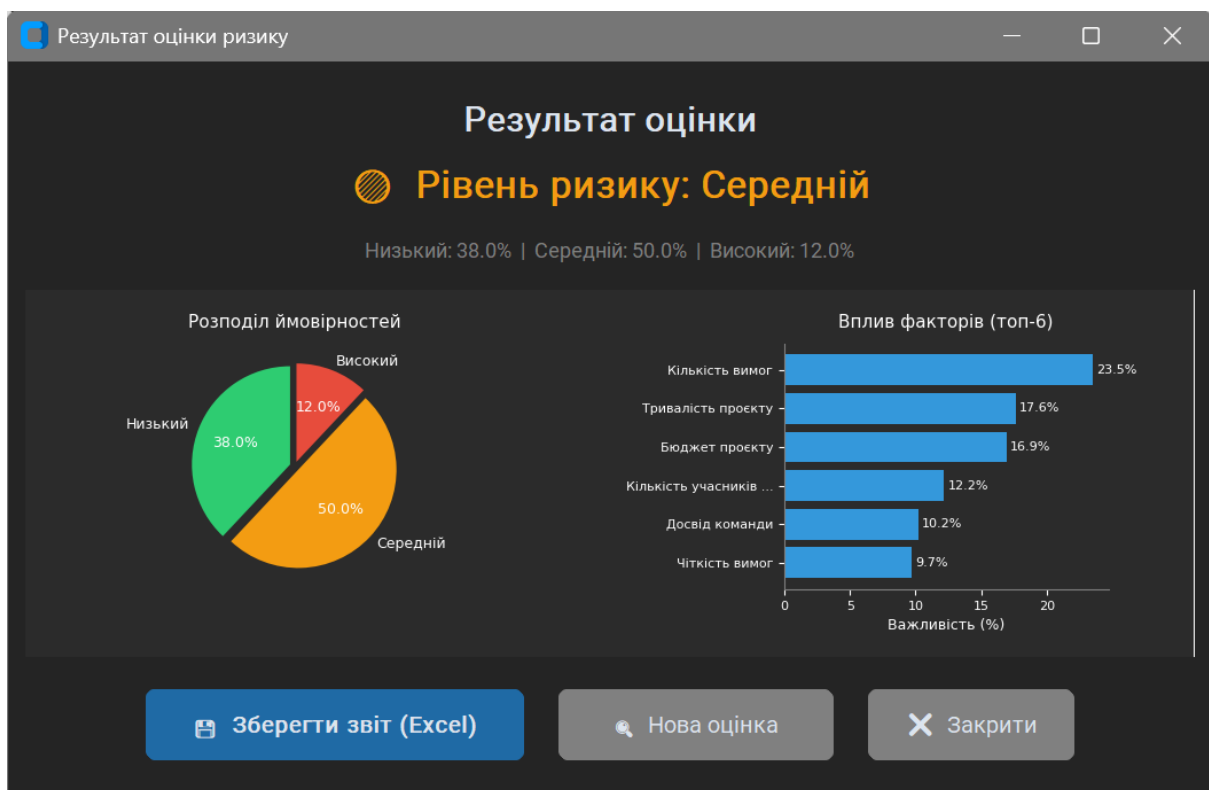
Параметр	Тип	Допустимий діапазон
Кількість учасників команди	Ціле число	1 – 50
Досвід команди (роки)	Дробове число	0.0 – 20.0
Бюджет проєкту (тис. грн)	Дробове число	10.0 – 10000.0
Тривалість проєкту (тижні)	Ціле число	1 – 104
Кількість вимог	Ціле число	1 – 500
Чіткість вимог (0–1)	Дробове число	0.0 – 1.0
Кількість стейкхолдерів	Ціле число	1 – 20
Наявність ризик-менеджера	Булеве значення	0 або 1

Кнопка "Оцінити ризик" — запускає валідацію всіх полів та передає дані в ML-модель. Якщо хоча б одне поле містить помилку — під ним відображається повідомлення і оцінка не виконується.

Кнопка "Очистити" — скидає всі поля до стандартних (дефолтних) значень та прибирає повідомлення про помилки.

5.3 Вікно результатів

Відображається після успішного введення даних та натискання "Оцінити ризик". Показує результат оцінки разом з двома графіками.



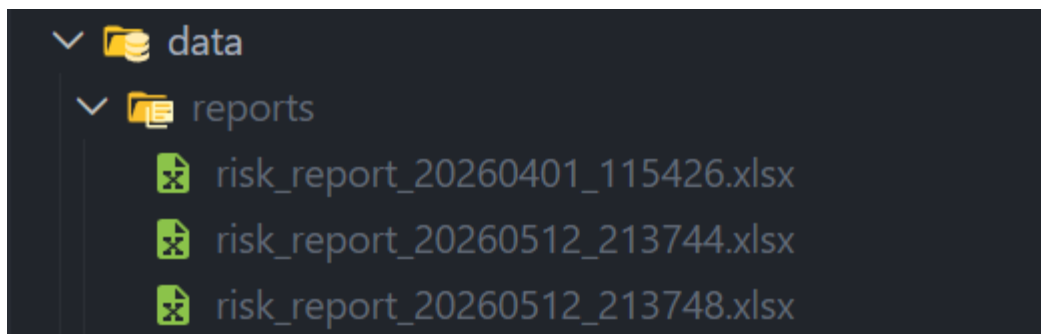
Елементи вікна результатів:

Рівень ризику — велика кольорова мітка у верхній частині вікна. Зелений колір означає низький ризик, помаранчевий — середній, червоний — високий.

Розподіл ймовірностей (Pie chart) — кругова діаграма що показує ймовірність кожного рівня ризику у відсотках.

Вплив факторів (Bar chart) — горизонтальна стовпчикова діаграма що показує які параметри найбільше вплинули на рішення моделі.

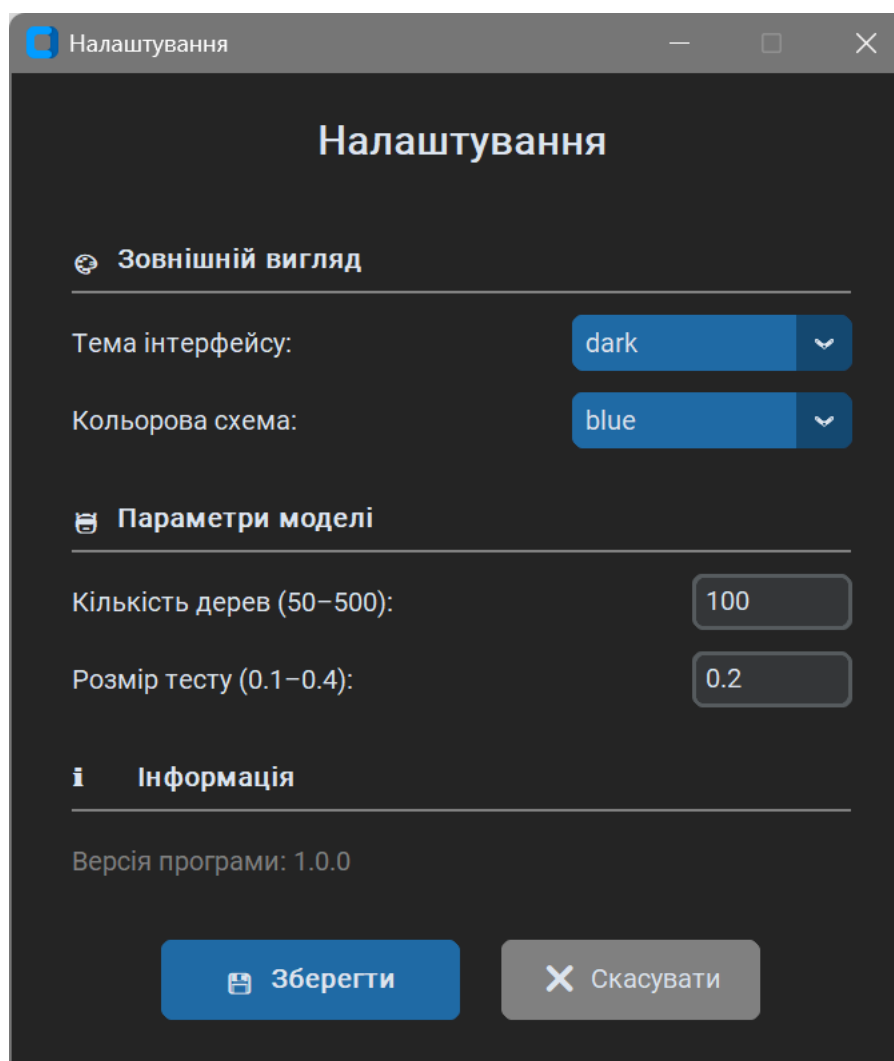
Кнопка "Зберегти звіт (Excel)" — генерує Excel-файл з повними результатами оцінки та зберігає його у папку `data/reports/`.



Кнопка "Нова оцінка" — закриває поточне вікно і відкриває форму для нової оцінки.

Кнопка "Закрити" — закриває вікно результатів і повертає до головного вікна.

5.4 Вікно налаштувань



Елементи вікна налаштувань:

Тема інтерфейсу — випадаючий список з опціями dark (темна), light (світла), system (системна). Зміна теми застосовується одразу після збереження.

Кольорова схема — випадаючий список з опціями blue, green, dark-blue. Визначає акцентний колір кнопок та елементів інтерфейсу.

Кількість дерев (50–500) — кількість дерев рішень у Random Forest моделі. Більше дерев — вища точність, але повільніше навчання.

Розмір тесту (0.1–0.4) — частка даних що використовується для тестування моделі (наприклад, 0.2 = 20% даних).

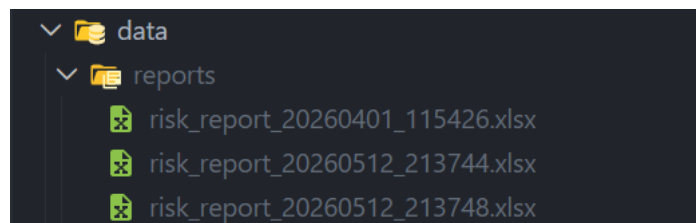
Кнопка "Зберегти" — зберігає налаштування у файл `config.json` та застосовує тему одразу.

Кнопка "Скасувати" — закриває вікно без збереження змін.

6. Огляд сценаріїв роботи

Сценарій 1 — Стандартна оцінка ризику проєкту

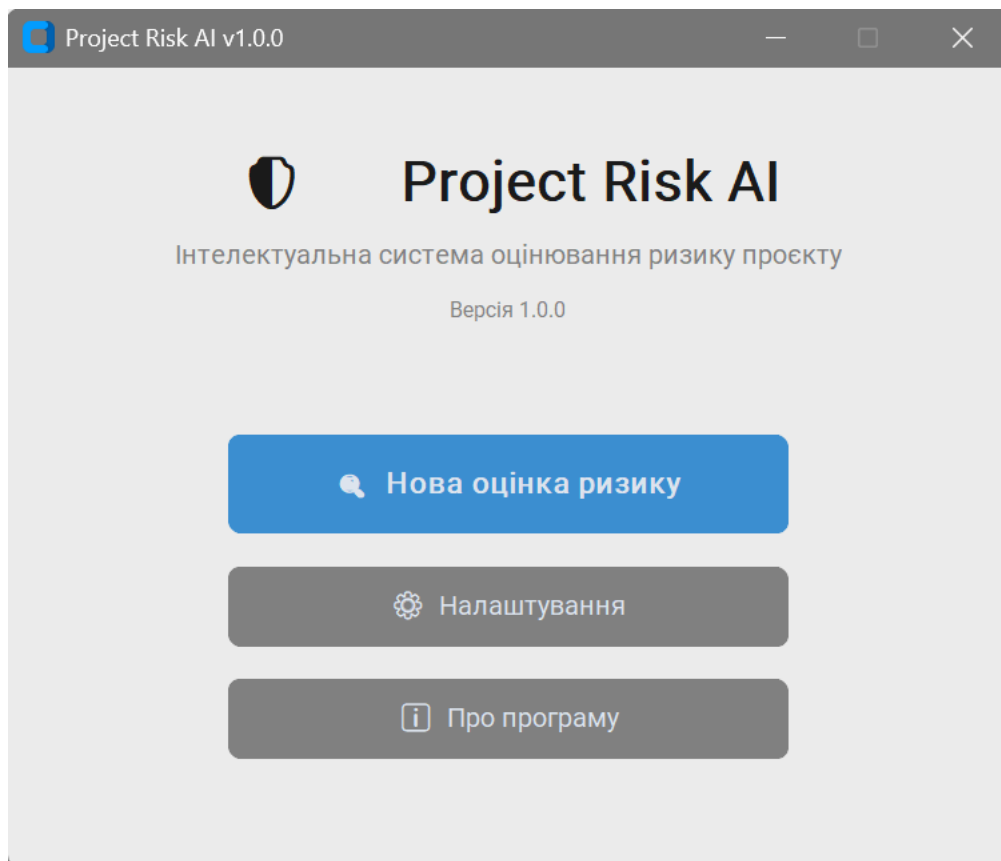
1. Запустити програму командою `python main.py`
2. Натиснути кнопку "Нова оцінка ризику"
3. Заповнити всі вісім полів форми
4. Натиснути "Оцінити ризик"
5. Переглянути результат та графіки
6. Натиснути "Зберегти звіт (Excel)" для збереження результатів
7. Знайти збережений файл у папці `data/reports/`



1	Звіт оцінки ризику проекту		
2	<i>Project Risk AI v1.0.0 01.04.2026 11:54</i>		
3			
4	РЕЗУЛЬТАТ ОЦІНКИ		
5	Рівень ризику:	Низький	
6			
7	Ймовірності по рівнях ризику		
8	Рівень ризику	Ймовірність	
9	Низький	47.0%	
10	Середній	35.0%	
11	Високий	18.0%	
12			
13	Введені параметри проекту		
14	Параметр	Значення	
15	Кількість учасників команди	10	
16	Досвід команди (роки)	2	
17	Бюджет проекту (тис. грн)	500	
18	Тривалість проекту (тижні)	12	
19	Кількість вимог	30	
20	Чіткість вимог (0–1)	0.7	
21	Кількість стейкхолдерів	3	
22	Наявність ризик-менеджера	1	
23			
24	Важливість факторів (вплив на ризик)		
25	Фактор	Важливість	
26	Кількість вимог	23.50%	
27	Тривалість проекту (тижні)	17.63%	
28	Бюджет проекту (тис. грн)	16.93%	
29	Кількість учасників команди	12.17%	
30	Досвід команди (роки)	10.18%	
31	Чіткість вимог (0–1)	9.67%	
32	Кількість стейкхолдерів	7.08%	
33	Наявність ризик-менеджера	2.84%	

Сценарій 2 — Зміна теми інтерфейсу

1. На головному вікні натиснути "Налаштування"
2. У списку "Тема інтерфейсу" вибрати light
3. Натиснути "Зберегти"
4. Переконатись що тема змінилась



7. Можливі помилки та їх усунення

Помилка	Причина	Рішення
ModuleNotFoundError при запуску	Не встановлені бібліотеки або не активовано venv	Активуйте venv та виконайте <code>pip install -r requirements.txt</code>
Поле підсвічується червоним	Введено некоректне значення	Перевірте підказку під полем і введіть значення у вказаному діапазоні
Кнопки у налаштуваннях не видно	Малий розмір екрану	Розтягніть вікно або прокрутіть вниз
Excel файл не відкривається	Файл вже відкритий в іншій програмі	Закрийте Excel та спробуйте знову

Помилка	Причина	Рішення
Програма довго запускається	Перший запуск — навчання моделі	Зачекайте 5-10 секунд, модель зберігається для наступних запусків

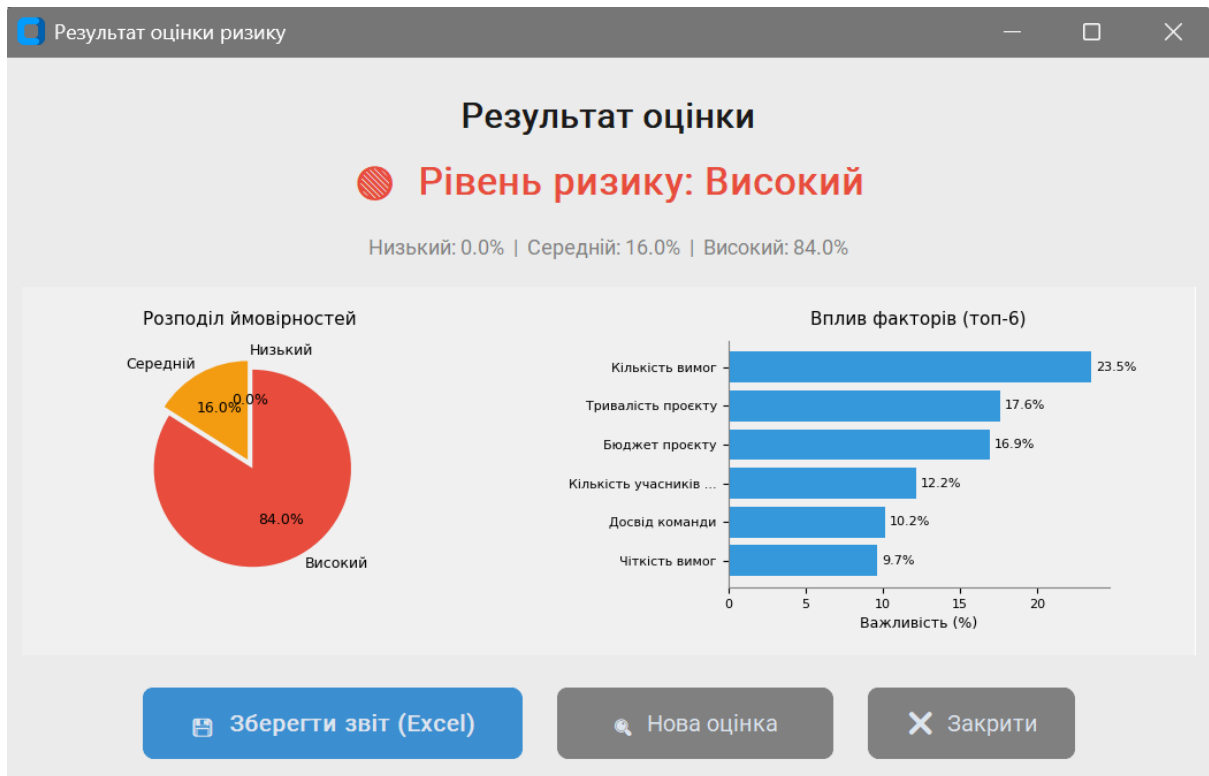
8. Приклад роботи з програмою

Приклад: Оцінка ризику малого проєкту з недосвідченою командою

Введіть наступні параметри:

- Кількість учасників: **3**
- Досвід команди: **0.5** роки
- Бюджет: **50** тис. грн
- Тривалість: **52** тижні
- Кількість вимог: **200**
- Чіткість вимог: **0.2**
- Стейкхолдерів: **15**
- Ризик-менеджер: **0** (відсутній)

Очікуваний результат: **Високий ризик** (ймовірність ~84%)

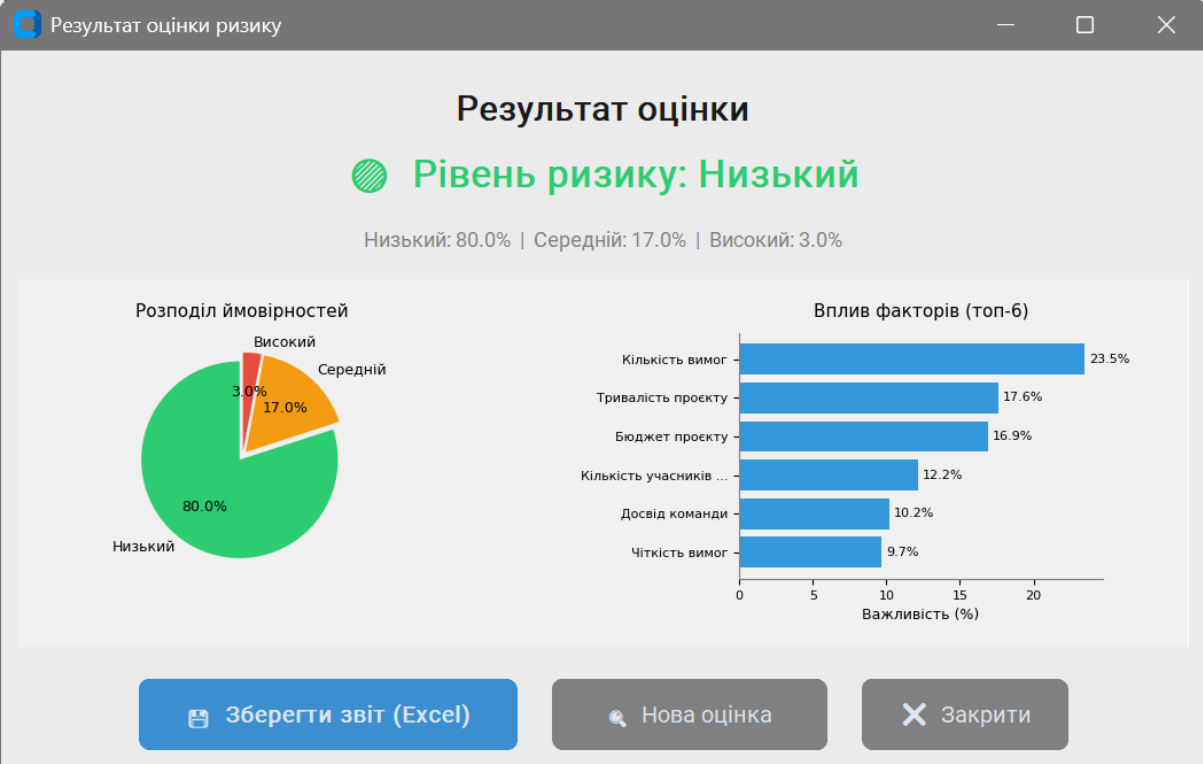


Приклад: Оцінка ризику великого досвідченого проєкту

Введіть наступні параметри:

- Кількість учасників: **20**
- Досвід команди: **8.0** роки
- Бюджет: **5000** тис. грн
- Тривалість: **12** тижні
- Кількість вимог: **30**
- Чіткість вимог: **0.9**
- Стейкхолдерів: **3**
- Ризик-менеджер: **1** (присутній)

Очікуваний результат: **Низький ризик**



СХЕМИ СТРУКТУР ДАНИХ

Схема 1 — Структура папок проекту

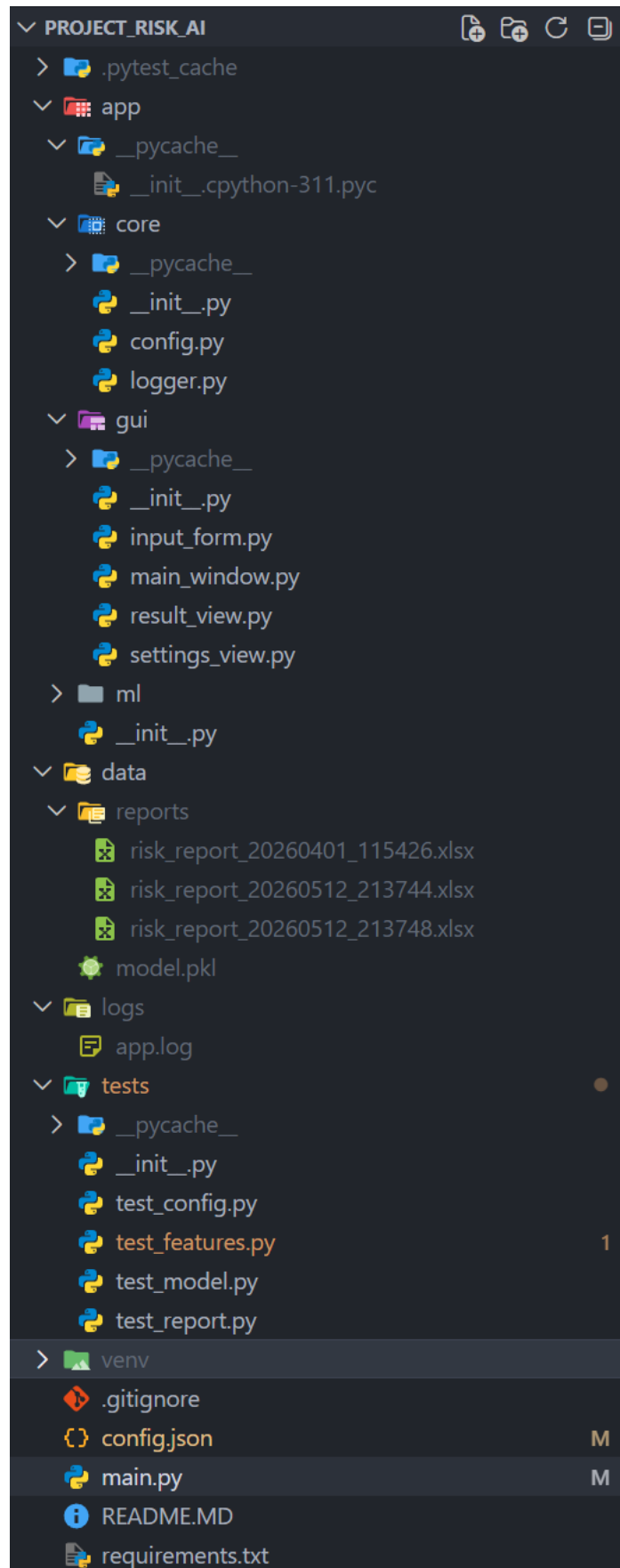
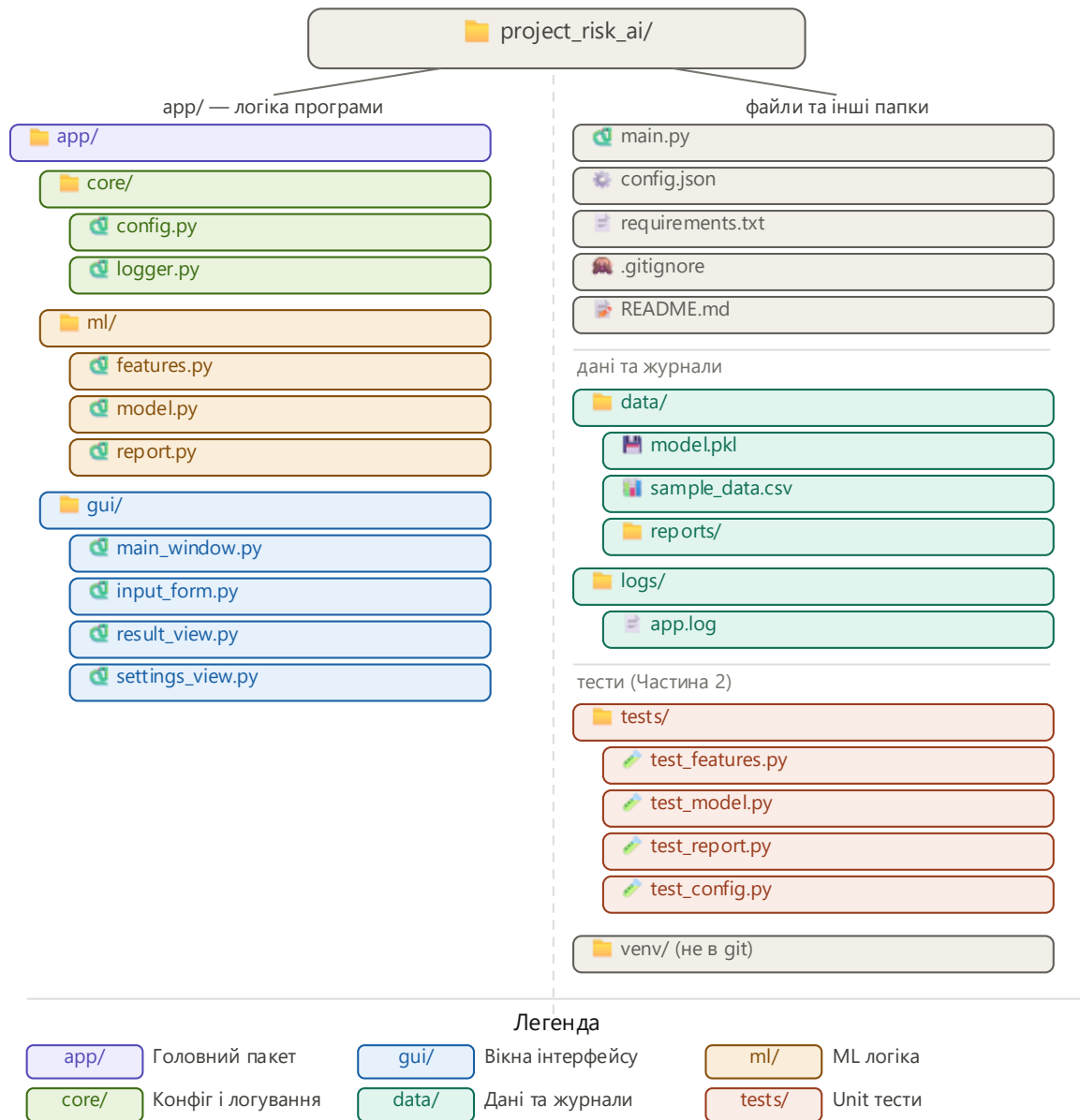


Схема 1 — Структура папок Project Risk AI v1.0.0

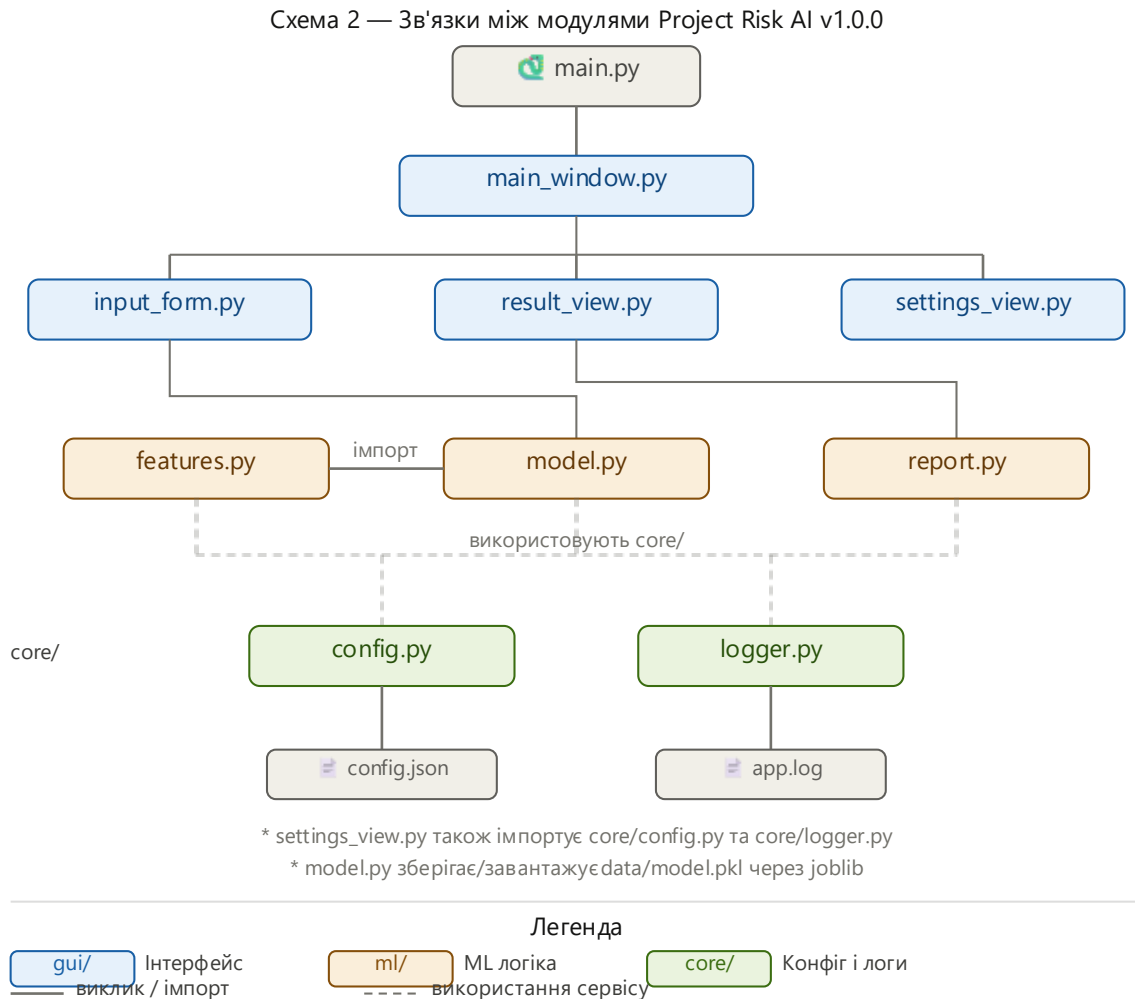


Опис структури: Проєкт організований за модульним принципом.

Папка **app/** містить весь код програми і розділена на три підмодулі: **core/** для базової інфраструктури, **ml/** для машинного навчання та **gui/** для графічного інтерфейсу.

Папка **data/** зберігає навчені моделі та згенеровані звіти. Папка **tests/** містить автоматизовані тести. Конфігурація програми винесена у зовнішній файл **config.json** в корені проєкту.

Схема 2 — Зв'язки між модулями



Опис зв'язків:

Точка входу main.py запускає головне вікно main_window.py.

Головне вікно відкриває три дочірні вікна: input_form.py (форма введення), result_view.py (результати) та settings_view.py (налаштування).

Форма введення викликає model.py для передбачення, а вікно результатів викликає report.py для генерації звіту.

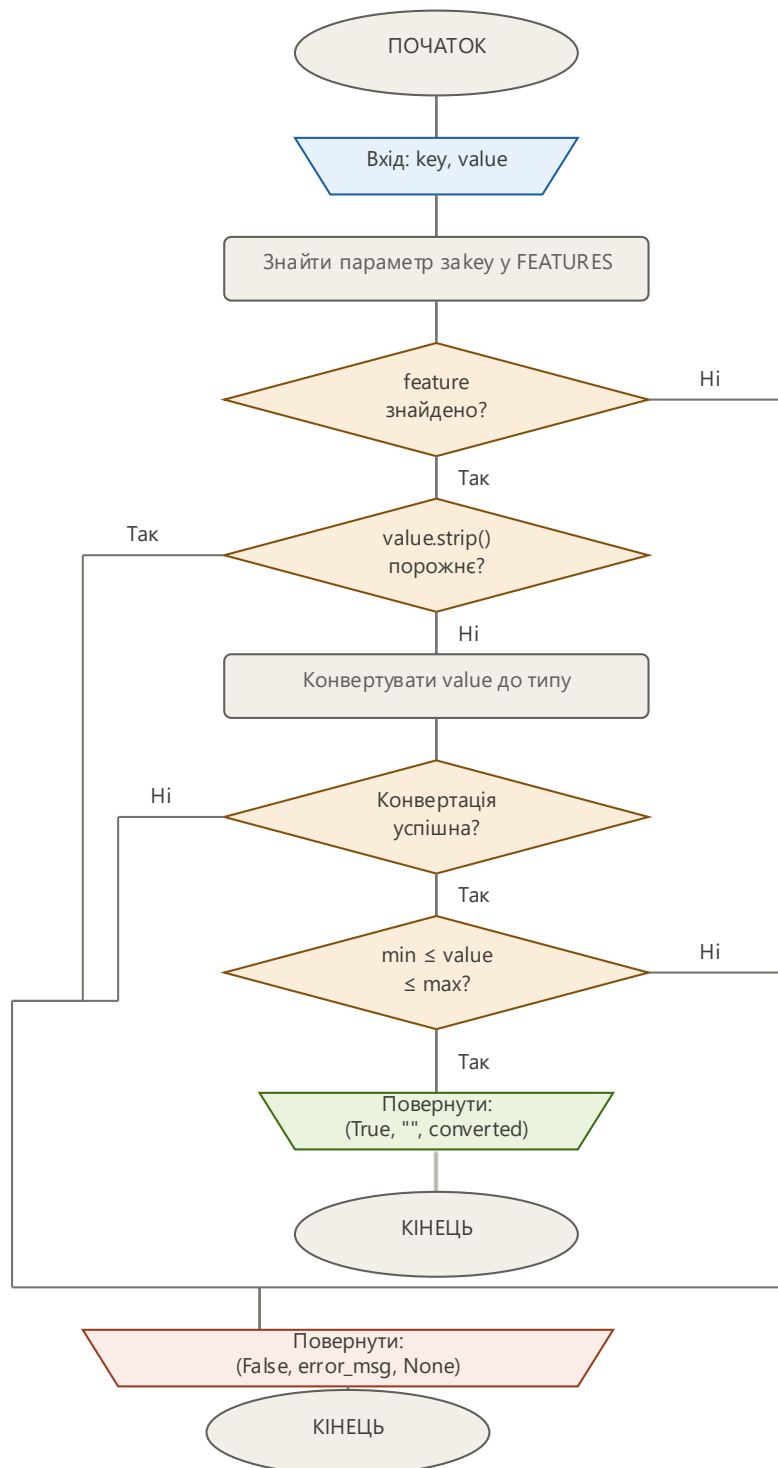
Обидва ML-модулі використовують features.py для отримання опису параметрів.

Всі модулі програми імпортують core/config.py для читання налаштувань та core/logger.py для запису логів.

БЛОК-СХЕМИ АЛГОРИТМІВ

Блок-схема 1 — Функція `validate_input()`

Блок-схема: `validate_input(key, value)`
Модуль `app/ml/features.py` — ДСТУ ISO 5807:2016

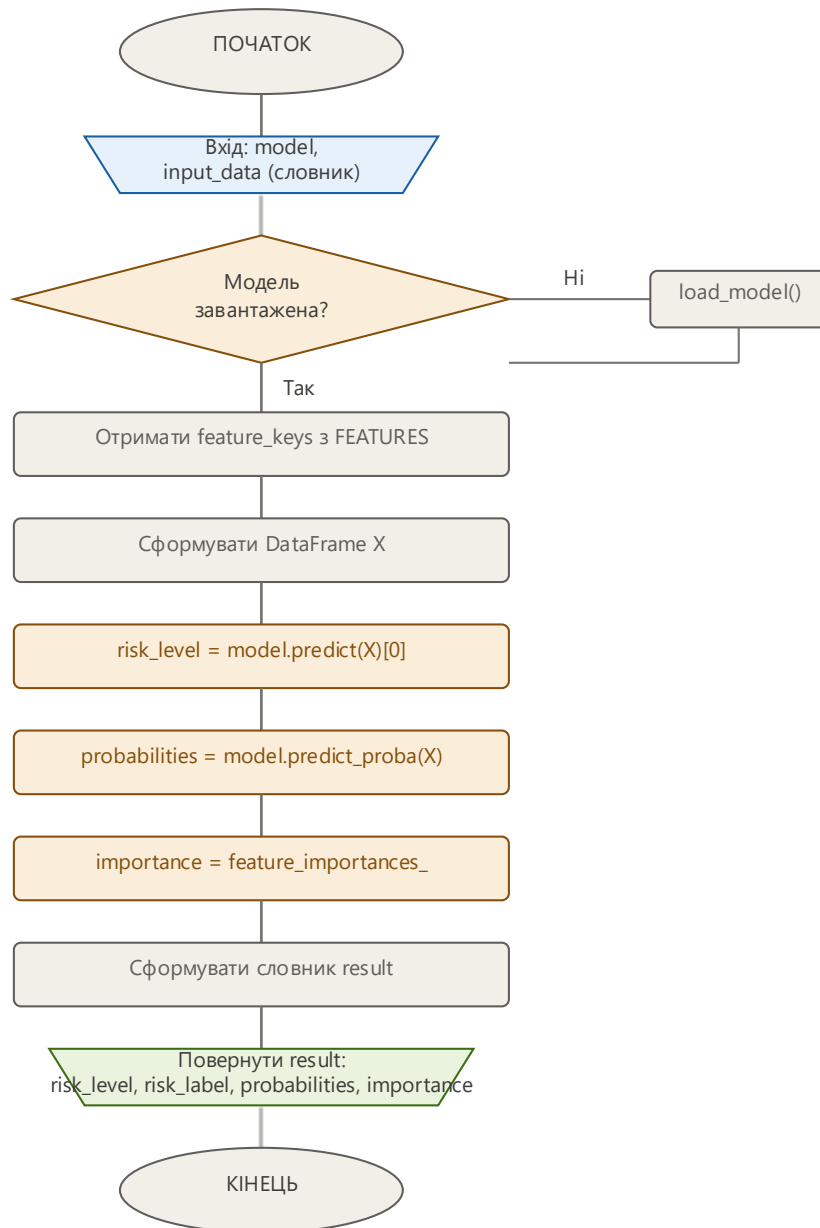


Опис алгоритму: Функція отримує ключ параметра та рядкове значення введене користувачем. Спочатку перевіряється чи існує параметр з таким ключем у списку FEATURES. Потім перевіряється чи поле не є порожнім. Далі виконується спроба

конвертувати рядок до відповідного типу (int, float або bool). Якщо конвертація успішна — перевіряється чи значення знаходиться у допустимому діапазоні. При успішному проходженні всіх перевірок повертається кортеж (True, "", converted_value). При будь-якій помилці — (False, error_message, None).

Блок-схема 2 — Функція `predict()`

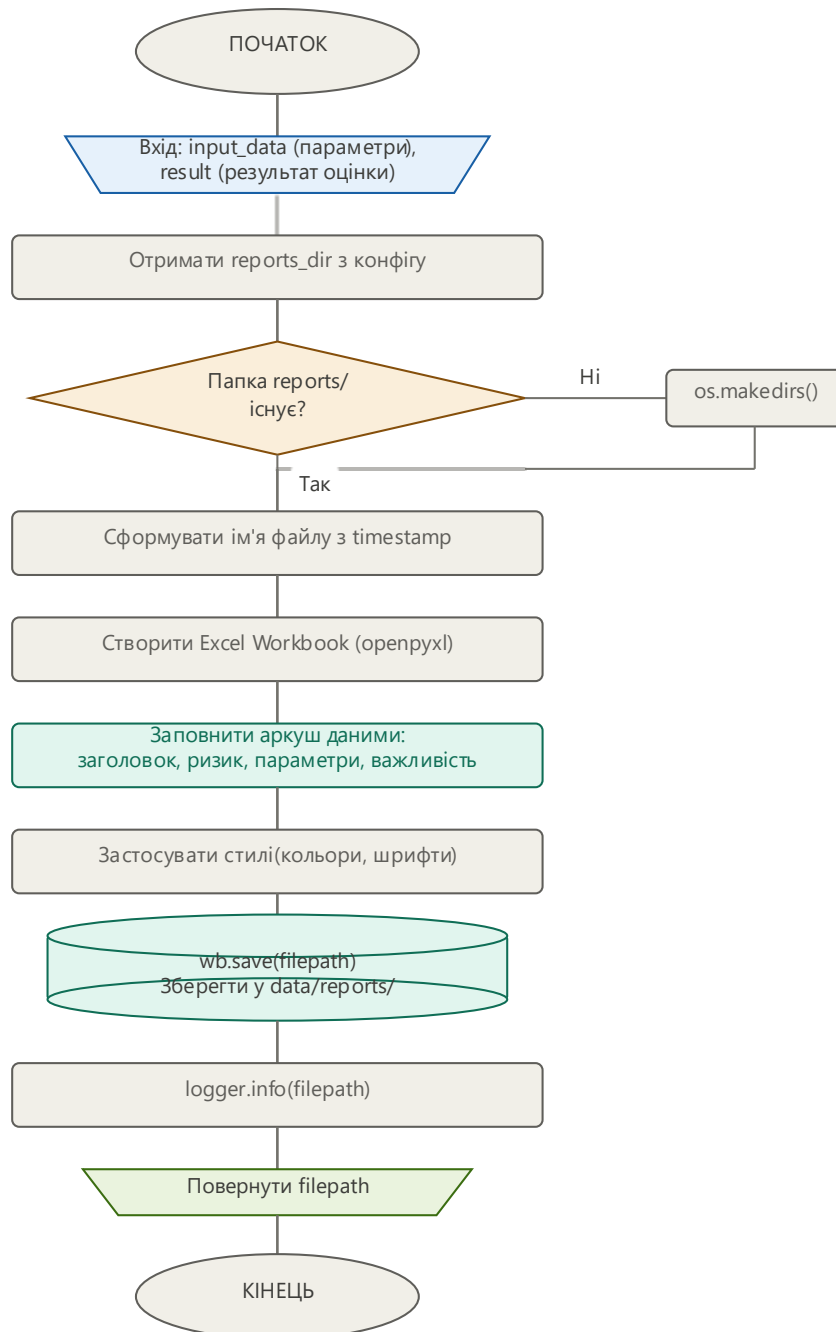
Блок-схема: `predict(model, input_data)`
Модуль `app/ml/model.py` — ДСТУ ISO 5807:2016



Опис алгоритму: Функція отримує навчену модель та словник з параметрами проєкту. Перевіряється чи модель завантажена, якщо ні — виконується автоматичне завантаження з файлу. Далі формується DataFrame з вхідних даних у форматі який очікує scikit-learn. Модель виконує передбачення класу ризику (0, 1 або 2) та ймовірностей для кожного класу. Також зчитується важливість кожного параметра з атрибуту `feature_importances_`. Всі результати збираються у словник і повертаються.

Блок-схема 3 — Функція `generate_report()`

Блок-схема: `generate_report(input_data, result)`
Модуль `app/ml/report.py` — ДСТУ ISO 5807:2016



Опис алгоритму: Функція отримує введені параметри та результати оцінки. Спочатку перевіряється існування папки для звітів і створюється якщо відсутня. Генерується унікальне ім'я файлу на основі поточного часу. Створюється новий Excel-файл через бібліотеку `openpyxl`. Аркуш заповнюється даними: заголовок звіту, рівень ризику з кольоровою заливкою, таблиця ймовірностей, таблиця введених параметрів та таблиця важливості факторів. Застосовуються стилі оформлення. Файл зберігається на диск і шлях до нього записується в лог та повертається як результат функції.

ЛІСТИНГ ПРОГРАМИ

main.py — Точка входу

Точка входу програми. Налаштовує шляхи імпорту, зчитує конфігурацію та запускає головне вікно.

```
# source venv/Scripts/activate
# pytest tests/ -v

import sys
import os

# Додаємо корінь проекту до шляху пошуку модулів
# Це потрібно щоб імпорти тину "from app.core..." працювали
коректно
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from app.core.logger import logger
from app.core.config import get
from app.gui.main_window import MainWindow

def main():
    """Головна функція запуску програми."""
    app_name = get("app.name", "Project Risk AI")
    version = get("app.version", "1.0.0")

    logger.info(f"Запуск програми {app_name} v{version}")

    try:
        app = MainWindow()
        app.mainloop()
    except Exception as e:
        logger.error(f"Критична помилка: {e}", exc_info=True)
        raise
    finally:
        logger.info("Програму завершено")

if __name__ == "__main__":
```

```
main()
```

app/core/config.py — Модуль конфігурації

Забезпечує завантаження та збереження конфігурації з файлу config.json. Функція get() дозволяє зручно отримувати вкладені значення через крапкову нотацію.

```
# app/core/config.py
# Модуль для роботи з конфігураційним файлом програми

import json
import os

# Шлях до config.json – відносно цього файлу піднімаємось на 2
# рівні вгору
CONFIG_PATH = os.path.join(
    os.path.dirname(__file__), "..", "..", "config.json"
)

def load_config() -> dict:
    """Завантажує конфігурацію з config.json."""
    config_path = os.path.abspath(CONFIG_PATH)

    if not os.path.exists(config_path):
        print(f"[WARNING] config.json не знайдено:
{config_path}")
        return {}

    with open(config_path, "r", encoding="utf-8") as f:
        return json.load(f)

def save_config(config: dict) -> None:
    """Зберігає оновлену конфігурацію назад у config.json."""
    config_path = os.path.abspath(CONFIG_PATH)

    with open(config_path, "w", encoding="utf-8") as f:
        json.dump(config, f, ensure_ascii=False, indent=2)

def get(key_path: str, default=None):
```

```

"""
Отримує значення з конфігу по точковому шляху.

Приклади:
    get("app.version")      -> "1.0.0"
    get("paths.log_file")  -> "logs/app.log"
    get("app.missing", 0)  -> 0
"""

config = load_config()
keys = key_path.split(".") # "app.version" -> ["app",
"version"]

value = config
for key in keys:
    if isinstance(value, dict) and key in value:
        value = value[key]
    else:
        return default

return value

```

app/core/logger.py — Модуль логування

Налаштовує систему логування з одночасним записом у файл logs/app.log та виведенням у термінал.

```

# app/core/logger.py
# Налаштування логування — пише в файл і термінал одночасно

import logging
import os
from app.core.config import get

def setup_logger() -> logging.Logger:
    """
    Створює головний логер програми.
    Логи йдуть одночасно в logs/app.log і термінал.
    """
    log_file = get("paths.log_file", "logs/app.log")

```

```
log_dir = os.path.dirname(log_file)

# Створюємо папку logs/ якщо не існує
if log_dir and not os.path.exists(log_dir):
    os.makedirs(log_dir)

log_format = "%(asctime)s | %(levelname)-8s | %(message)s"
date_format = "%Y-%m-%d %H:%M:%S"

logger = logging.getLogger("ProjectRiskAI")
logger.setLevel(logging.DEBUG)

# Уникаємо дублювання якщо logger вже створений
if logger.handlers:
    return logger

# Handler 1: запис у файл
file_handler = logging.FileHandler(log_file, encoding="utf-8")
file_handler.setLevel(logging.DEBUG)
file_handler.setFormatter(logging.Formatter(log_format,
date_format))

# Handler 2: вивід у термінал
console_handler = logging.StreamHandler()
console_handler.setLevel(logging.INFO)
console_handler.setFormatter(logging.Formatter(log_format,
date_format))

logger.addHandler(file_handler)
logger.addHandler(console_handler)

logger.info("Логер ініціалізовано успішно")
return logger

# Глобальний логер – інші модулі імпортують його так:
# from app.core.Logger import logger
logger = setup_logger()
```

app/ml/features.py — Опис параметрів та валідація

Містить опис всіх восьми вхідних параметрів моделі з їх типами та допустимими діапазонами. Функція `validate_input()` перевіряє кожне введене значення.

```
# app/ml/features.py
# Опис вхідних параметрів моделі оцінки ризику проєкту

# Список всіх параметрів які вводить користувач
# Кожен параметр – словник з описом для GUI та валідації
FEATURES = [
    {
        "key": "team_size",
        "label": "Кількість учасників команди",
        "type": "int",
        "min": 1,
        "max": 50,
        "default": 5,
        "hint": "Введіть кількість людей у команді (1-50)"
    },
    {
        "key": "team_experience",
        "label": "Досвід команди (роки)",
        "type": "float",
        "min": 0.0,
        "max": 20.0,
        "default": 2.0,
        "hint": "Середній досвід учасників у роках (0-20)"
    },
    {
        "key": "budget",
        "label": "Бюджет проєкту (тис. грн)",
        "type": "float",
        "min": 10.0,
        "max": 10000.0,
        "default": 500.0,
        "hint": "Загальний бюджет проєкту в тисячах гривень (10-10000)"
    },
    {
        "key": "duration_weeks",
```

```
    "label": "Тривалість проєкту (тижні)",
    "type": "int",
    "min": 1,
    "max": 104,
    "default": 12,
    "hint": "Планована тривалість проєкту в тижнях (1-104)"
  },
  {
    "key": "requirements_count",
    "label": "Кількість вимог",
    "type": "int",
    "min": 1,
    "max": 500,
    "default": 30,
    "hint": "Загальна кількість функціональних вимог (1-500)"
  },
  {
    "key": "requirements_clarity",
    "label": "Чіткість вимог (0-1)",
    "type": "float",
    "min": 0.0,
    "max": 1.0,
    "default": 0.7,
    "hint": "Наскільки чіткі вимоги: 0 = повна невизначеність, 1 = повна ясність"
  },
  {
    "key": "stakeholders_count",
    "label": "Кількість стейкхолдерів",
    "type": "int",
    "min": 1,
    "max": 20,
    "default": 3,
    "hint": "Кількість зацікавлених сторін проєкту (1-20)"
  },
  {
    "key": "has_risk_manager",
    "label": "Наявність ризик-менеджера",
    "type": "bool",
```

```

        "min": 0,
        "max": 1,
        "default": 0,
        "hint": "Чи є в команді відповідальний за управління
ризиками?"
    },
]

# Назви рівнів ризику – використовуються в GUI і звіті
RISK_LABELS = {
    0: "Низький",
    1: "Середній",
    2: "Високий"
}

# Кольори для відображення рівня ризику в GUI
RISK_COLORS = {
    0: "#2ecc71",    # зелений
    1: "#f39c12",    # помаранчевий
    2: "#e74c3c"     # червоний
}

def validate_input(key: str, value: str) -> tuple[bool, str,
float | int | None]:
    """
    Валідує введені користувачем значення для конкретного
    параметра.

    Повертає tuple з трьох елементів:
        (is_valid, error_message, converted_value)

    Приклади:
        validate_input("team_size", "5")    -> (True, "", 5)
        validate_input("team_size", "abc")  -> (False, "Введіть
ціле число", None)
        validate_input("team_size", "999") -> (False, "Значення
має бути від 1 до 50", None)
    """
    # Знаходимо опис параметра за ключем

```



```

    feature = next((f for f in FEATURES if f["key"] == key),
None)

    if feature is None:
        return False, f"Невідомий параметр: {key}", None

    # Перевірка на порожнє поле
    if value.strip() == "":
        return False, "Поле не може бути порожнім", None

    # Перевірка типу і конвертація
    try:
        if feature["type"] == "int":
            converted = int(value)
        elif feature["type"] == "float":
            converted = float(value.replace(",", ".")) #
підтримка коми як роздільника
        elif feature["type"] == "bool":
            converted = int(value)
            if converted not in (0, 1):
                return False, "Введіть 0 або 1", None
        else:
            converted = float(value)
    except ValueError:
        if feature["type"] == "int":
            return False, "Введіть ціле число", None
        else:
            return False, "Введіть числове значення", None

    # Перевірка діапазону
    if converted < feature["min"] or converted >
feature["max"]:
        return False, f"Значення має бути від {feature['min']}
до {feature['max']}", None

    return True, "", converted

def get_feature_keys() -> list[str]:

```

```
"""Повертає список ключів всіх параметрів – для формування
масиву даних."""
return [f["key"] for f in FEATURES]
```

app/ml/model.py — ML модуль

Генерує синтетичні тренувальні дані, навчає модель Random Forest та виконує передбачення ризику для нових даних.

```
# app/ml/model.py
# ML модуль: генерація даних, навчання і передбачення ризику
проекту

import os
import numpy as np
import pandas as pd
import joblib

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,
classification_report

from app.core.config import get
from app.core.logger import logger
from app.ml.features import get_feature_keys, RISK_LABELS

def generate_training_data(n_samples: int = 1000) ->
pd.DataFrame:
    """
    Генерує синтетичні тренувальні дані для моделі.

    В реальному проєкті тут були б реальні дані з БД або CSV.
    Ми генеруємо дані з логікою: більша команда + більший
    досвід +
    більший бюджет + чіткі вимоги = менший ризик.

    Повертає DataFrame з колонками-параметрами + колонка
    'risk_level'.
```

```

"""
np.random.seed(get("model.random_state", 42))
n = n_samples

# Генеруємо випадкові значення для кожного параметра
data = {
    "team_size": np.random.randint(1, 51, n),
    "team_experience": np.round(np.random.uniform(0,
20, n), 1),
    "budget": np.round(np.random.uniform(10,
10000, n), 1),
    "duration_weeks": np.random.randint(1, 105, n),
    "requirements_count": np.random.randint(1, 501, n),
    "requirements_clarity": np.round(np.random.uniform(0,
1, n), 2),
    "stakeholders_count": np.random.randint(1, 21, n),
    "has_risk_manager": np.random.randint(0, 2, n),
}

df = pd.DataFrame(data)

# Розраховуємо "ризик" на основі логіки предметної області
# risk_score: більше = гірше
risk_score = (
    (50 - df["team_size"]) * 0.3 + # менша
команда = більший ризик
    (20 - df["team_experience"]) * 0.5 + # менший
досвід = більший ризик
    (10000 - df["budget"]) / 500 + # менший
бюджет = більший ризик
    df["duration_weeks"] * 0.2 + # довший
проект = більший ризик
    df["requirements_count"] * 0.05 + # більше
вимог = більший ризик
    (1 - df["requirements_clarity"]) * 10 + # нечіткі
вимоги = більший ризик
    df["stakeholders_count"] * 0.3 + # більше
стейкхолдерів = більший ризик
    (1 - df["has_risk_manager"]) * 5 # немає
ризик-менеджера = більший ризик

```

```

)

# Додаємо невеликий шум щоб дані не були ідеальними
noise = np.random.normal(0, 2, n)
risk_score += noise

# Конвертуємо числовий score в категорії 0/1/2
# за допомогою перцентилів – рівномірний розподіл класів
p33 = np.percentile(risk_score, 33)
p66 = np.percentile(risk_score, 66)

df["risk_level"] = np.where(
    risk_score <= p33, 0,          # Низький ризик
    np.where(risk_score <= p66, 1, 2) # Середній або
Високий
)

logger.info(f"Згенеровано {n} тренувальних зразків")
return df

def train_model(df: pd.DataFrame = None) -> tuple:
    """
    Навчає RandomForest модель на тренувальних даних.

    Якщо df не передано – генерує дані автоматично.
    Зберігає навчену модель у файл model.pkl.

    Повертає (model, accuracy, report) – модель, точність і
    звіт.
    """
    if df is None:
        df = generate_training_data()

    feature_keys = get_feature_keys()
    X = df[feature_keys]    # вхідні параметри
    y = df["risk_level"]    # цільова змінна (0/1/2)

    # Розділяємо на тренувальну і тестову вибірки
    test_size = get("model.test_size", 0.2)

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=test_size,
    random_state=get("model.random_state", 42)
)

# Навчаємо Random Forest
n_estimators = get("model.n_estimators", 100)
model = RandomForestClassifier(
    n_estimators=n_estimators,
    random_state=get("model.random_state", 42)
)
model.fit(X_train, y_train)

# Оцінюємо точність
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(
    y_test, y_pred,
    target_names=[RISK_LABELS[i] for i in range(3)]
)

logger.info(f"Модель навчена. Точність: {accuracy:.2%}")

# Зберігаємо модель у файл
model_path = get("paths.model_file", "data/model.pkl")
os.makedirs(os.path.dirname(model_path), exist_ok=True)
joblib.dump(model, model_path)
logger.info(f"Модель збережена: {model_path}")

return model, accuracy, report

def load_model():
    """
    Завантажує збережену модель з файлу.
    Якщо файл не існує – навчає нову модель автоматично.
    """
    model_path = get("paths.model_file", "data/model.pkl")

```

```

    if os.path.exists(model_path):
        model = joblib.load(model_path)
        logger.info(f"Модель завантажена з файлу:
{model_path}")
        return model
    else:
        # Модель ще не існує – навчаємо першу
        logger.warning("Файл моделі не знайдено. Навчаємо нову
модель...")
        model, __, __ = train_model()
        return model

def predict(model, input_data: dict) -> dict:
    """
    Робить передбачення ризику для введених параметрів проєкту.

    input_data – словник з ключами як у FEATURES:
        {"team_size": 5, "team_experience": 2.0, ...}

    Повертає словник з результатами:
        {
            "risk_level": 1,
            "risk_label": "Середній",
            "probabilities": {"Низький": 0.2, "Середній": 0.6,
"Високий": 0.2},
            "feature_importance": {"team_size": 0.15, ...}
        }
    """
    feature_keys = get_feature_keys()

    # Формуємо DataFrame з одного рядка для передбачення
    X = pd.DataFrame([{"key": input_data[key] for key in
feature_keys}])

    # Передбачення класу і ймовірностей
    risk_level = int(model.predict(X)[0])
    probabilities = model.predict_proba(X)[0]

    # Важливість кожного параметра для цього передбачення

```

```

        importance = dict(zip(feature_keys,
model.feature_importances_))

    result = {
        "risk_level": risk_level,
        "risk_label": RISK_LABELS[risk_level],
        "probabilities": {
            RISK_LABELS[i]: round(float(probabilities[i]), 3)
            for i in range(len(probabilities))
        },
        "feature_importance": {
            k: round(v, 4) for k, v in
            sorted(importance.items(), key=lambda x: x[1],
reverse=True)
        }
    }

    logger.info(f"Передбачення: {result['risk_label']} (рівень
{risk_level})")
    return result

```

app/ml/report.py — Генерація звітів

Створює структурований Excel-звіт з результатами оцінки ризику з використанням стилізованих таблиць.

```

# app/ml/report.py
# Генерація Excel звіту з результатами оцінки ризику проєкту

import os
from datetime import datetime

import openpyxl
from openpyxl.styles import (
    Font, PatternFill, Alignment, Border, Side
)

from app.core.config import get
from app.core.logger import logger
from app.ml.features import FEATURES, RISK_COLORS

```

```

def _get_risk_fill(risk_level: int) -> PatternFill:
    """Повертає заливку клітинки відповідно до рівня ризику."""
    # Прибираємо # з кольору і конвертуємо для openpyxl
    color_map = {
        0: "2ecc71", # зелений
        1: "f39c12", # помаранчевий
        2: "e74c3c"  # червоний
    }
    color = color_map.get(risk_level, "ffffff")
    return PatternFill(start_color=color, end_color=color,
fill_type="solid")

def _thin_border() -> Border:
    """Повертає тонку рамку для клітинок таблиці."""
    thin = Side(style="thin")
    return Border(left=thin, right=thin, top=thin, bottom=thin)

def generate_report(input_data: dict, result: dict) -> str:
    """
    Генерує Excel звіт з результатами оцінки ризику.

    input_data – словник введених параметрів проєкту
    result      – словник результатів від predict()

    Повертає шлях до збереженого файлу.
    """
    # Створюємо папку для звітів якщо не існує
    reports_dir = get("paths.reports_dir", "data/reports")
    os.makedirs(reports_dir, exist_ok=True)

    # Ім'я файлу містить дату і час – кожен звіт унікальний
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"risk_report_{timestamp}.xlsx"
    filepath = os.path.join(reports_dir, filename)

    # Створюємо новий Excel файл
    wb = openpyxl.Workbook()

```



```

ws = wb.active
ws.title = "Оцінка ризику"

# --- Стили ---
header_font = Font(bold=True, size=12, color="FFFFFF")
header_fill = PatternFill(start_color="2c3e50",
end_color="2c3e50", fill_type="solid")
title_font = Font(bold=True, size=14)
center = Alignment(horizontal="center", vertical="center")
border = _thin_border()

# --- Заголовок звіту ---
ws.merge_cells("A1:C1")
ws["A1"] = f"Звіт оцінки ризику проєкту"
ws["A1"].font = Font(bold=True, size=16)
ws["A1"].alignment = center

ws.merge_cells("A2:C2")
app_name = get("app.name", "Project Risk AI")
version = get("app.version", "1.0.0")
ws["A2"] = f"{app_name} v{version} |
{datetime.now().strftime('%d.%m.%Y %H:%M')}"
ws["A2"].alignment = center
ws["A2"].font = Font(italic=True, color="7f8c8d")

ws.append([]) # порожній рядок

# --- Блок: Результат оцінки ---
ws.append(["РЕЗУЛЬТАТ ОЦІНКИ"])
ws[f"A{ws.max_row}"].font = Font(bold=True, size=12)

# Рівень ризику з кольоровою заливкою
risk_row = ws.max_row + 1
ws.cell(risk_row, 1, "Рівень ризику:")
ws.cell(risk_row, 1).font = Font(bold=True)

ws.cell(risk_row, 2, result["risk_label"])
ws.cell(risk_row, 2).font = Font(bold=True, size=12)
ws.cell(risk_row, 2).fill =
_get_risk_fill(result["risk_level"])

```

```

ws.cell(risk_row, 2).alignment = center
ws.cell(risk_row, 2).border = border

ws.append([]) # порожній рядок

# --- Блок: Ймовірності ---
ws.append(["Ймовірності по рівнях ризику"])
ws[f"A{ws.max_row}"].font = Font(bold=True)

# Заголовки таблиці ймовірностей
prob_header_row = ws.max_row + 1
headers = ["Рівень ризику", "Ймовірність", ""]
for col, header in enumerate(headers, 1):
    cell = ws.cell(prob_header_row, col, header)
    cell.font = header_font
    cell.fill = header_fill
    cell.alignment = center
    cell.border = border

# Дані ймовірностей
for label, prob in result["probabilities"].items():
    row = ws.max_row + 1
    ws.cell(row, 1, label).border = border
    ws.cell(row, 2, f"{prob * 100:.1f}%").border = border
    ws.cell(row, 2).alignment = center

ws.append([]) # порожній рядок

# --- Блок: Введені параметри ---
ws.append(["Введені параметри проєкту"])
ws[f"A{ws.max_row}"].font = Font(bold=True)

# Заголовки таблиці параметрів
param_header_row = ws.max_row + 1
for col, header in enumerate(["Параметр", "Значення", ""],
1):
    cell = ws.cell(param_header_row, col, header)
    cell.font = header_font
    cell.fill = header_fill
    cell.alignment = center

```

```

        cell.border = border

    # Дані параметрів – беремо label з FEATURES для зрозумілих
    назв
    for feature in FEATURES:
        key = feature["key"]
        label = feature["label"]
        value = input_data.get(key, "-")

        row = ws.max_row + 1
        ws.cell(row, 1, label).border = border
        ws.cell(row, 2, value).border = border
        ws.cell(row, 2).alignment = center

    ws.append([]) # порожній рядок

    # --- Блок: Важливість факторів ---
    ws.append(["Важливість факторів (вплив на ризик)"])
    ws[f"A{ws.max_row}"].font = Font(bold=True)

    imp_header_row = ws.max_row + 1
    for col, header in enumerate(["Фактор", "Важливість", ""],
1):
        cell = ws.cell(imp_header_row, col, header)
        cell.font = header_font
        cell.fill = header_fill
        cell.alignment = center
        cell.border = border

    # feature_importance вже відсортований за спаданням у
    model.py
    feature_labels = {f["key"]: f["label"] for f in FEATURES}
    for key, importance in
result["feature_importance"].items():
        row = ws.max_row + 1
        ws.cell(row, 1, feature_labels.get(key, key)).border =
border
        ws.cell(row, 2, f"{importance * 100:.2f}%").border =
border
        ws.cell(row, 2).alignment = center

```

```
# Ширина колонок
ws.column_dimensions["A"].width = 40
ws.column_dimensions["B"].width = 20
ws.column_dimensions["C"].width = 10

# Зберігаємо файл
wb.save(filepath)
logger.info(f"Звіт збережено: {filepath}")

return filepath
```

app/gui/main_window.py — Головне вікно

Реалізує головне вікно програми з кнопками навігації та логікою відкриття дочірніх вікон.

```
# app/gui/main_window.py
# Головне вікно програми

import customtkinter as ctk
from app.core.config import get
from app.core.logger import logger

class MainWindow(ctk.CTk):
    """
    Головне вікно програми.
    Наслідує CTk – це базовий клас вікна у customtkinter.
    """

    def __init__(self):
        super().__init__()

        # Застосовуємо тему з конфігу
        theme = get("appearance.theme", "dark")
        color = get("appearance.color_scheme", "blue")
        ctk.set_appearance_mode(theme)
        ctk.set_default_color_theme(color)
```

```

# Налаштування вікна
app_name = get("app.name", "Project Risk AI")
version = get("app.version", "1.0.0")
self.title(f"{app_name} v{version}")
self.geometry("500x400")
self.resizable(False, False)

# Центруємо вікно на екрані
self._center_window(500, 400)

logger.info("Головне вікно ініціалізовано")
self._build_ui()

def _center_window(self, width: int, height: int):
    """Розміщує вікно по центру екрану."""
    screen_w = self.winfo_screenwidth()
    screen_h = self.winfo_screenheight()
    x = (screen_w - width) // 2
    y = (screen_h - height) // 2
    self.geometry(f"{width}x{height}+{x}+{y}")

def _build_ui(self):
    """Будує всі елементи інтерфейсу головного вікна."""

    # --- Заголовок ---
    title_frame = ctk.CTkFrame(self,
fg_color="transparent")
    title_frame.pack(pady=(40, 10))

    ctk.CTkLabel(
        title_frame,
        text="🛡️ Project Risk AI",
        font=ctk.CTkFont(size=28, weight="bold")
    ).pack()

    ctk.CTkLabel(
        title_frame,
        text="Інтелектуальна система оцінювання ризику
проєкту",
        font=ctk.CTkFont(size=13),

```

```
        text_color="gray"
    ).pack(pady=(5, 0))

version = get("app.version", "1.0.0")
ctk.CTkLabel(
    title_frame,
    text=f"Версія {version}",
    font=ctk.CTkFont(size=11),
    text_color="gray"
).pack()

# --- Кнопки навігації ---
btn_frame = ctk.CTkFrame(self, fg_color="transparent")
btn_frame.pack(pady=30)

# Кнопка нової оцінки – головна дія
ctk.CTkButton(
    btn_frame,
    text="🔍 Нова оцінка ризику",
    font=ctk.CTkFont(size=15, weight="bold"),
    width=280,
    height=50,
    command=self._open_input_form
).pack(pady=8)

# Кнопка налаштувань
ctk.CTkButton(
    btn_frame,
    text="⚙️ Налаштування",
    font=ctk.CTkFont(size=13),
    width=280,
    height=40,
    fg_color="gray",
    hover_color="#555555",
    command=self._open_settings
).pack(pady=8)

# Кнопка "Про програму"
ctk.CTkButton(
    btn_frame,
```

```

        text="[i] Про програму",
        font=ctk.CTkFont(size=13),
        width=280,
        height=40,
        fg_color="gray",
        hover_color="#555555",
        command=self._show_about
    ).pack(pady=8)

# --- Футер ---
ctk.CTkLabel(
    self,
    text="© 2026 Project Risk AI",
    font=ctk.CTkFont(size=10),
    text_color="gray"
).pack(side="bottom", pady=10)

def _open_input_form(self):
    """Відкриває форму введення параметрів проєкту."""
    # Імпортуємо тут щоб уникнути циклічних імпортів
    from app.gui.input_form import InputForm
    logger.info("Відкрито форму введення параметрів")
    InputForm(self)

def _open_settings(self):
    """Відкриває вікно налаштувань."""
    from app.gui.settings_view import SettingsView
    logger.info("Відкрито налаштування")
    SettingsView(self)

def _show_about(self):
    """Показує діалог з інформацією про програму."""
    app_name = get("app.name", "Project Risk AI")
    version = get("app.version", "1.0.0")

    # CTkMessageBox – простий діалог через стандартний
tkinter
    import tkinter.messagebox as mb
    mb.showinfo(
        "Про програму",

```

```
f"{app_name}\n"
f"Версія: {version}\n\n"
f"Система оцінювання ризику зриву термінів\n"
f"IT-проєктів на основі машинного навчання.\n\n"
f"Алгоритм: Random Forest Classifier"
)
logger.info("Показано вікно 'Про програму'")
```

app/gui/input_form.py — Форма введення

Динамічно генерує форму на основі списку параметрів з features.py. Реалізує валідацію з відображенням помилок під кожним полем.

```
# app/gui/input_form.py
# Форма введення параметрів проєкту з валідацією

import customtkinter as ctk
import tkinter.messagebox as mb

from app.core.logger import logger
from app.ml.features import FEATURES, validate_input
from app.ml.model import load_model, predict

class InputForm(ctk.CTkToplevel):
    """
    Вікно форми введення параметрів проєкту.
    CTkToplevel – дочірнє вікно поверх головного.
    """

    def __init__(self, parent):
        super().__init__(parent)

        self.title("Нова оцінка ризику")
        self.geometry("560x700")
        self.resizable(False, True)
        self._center_window(560, 700)

        # Робимо вікно модальним – блокує головне вікно
        self.grab_set()
```



```

self.focus_set()

# Завантажуємо модель при відкритті форми
self.model = load_model()

# Словник для зберігання полів вводу {key: CTkEntry}
self.entries = {}

# Словник для зберігання міток помилок {key: CTkLabel}
self.error_labels = {}

logger.info("Форму введення відкрито")
self._build_ui()

def _center_window(self, width: int, height: int):
    """Центрує вікно на екрані."""
    screen_w = self.winfo_screenwidth()
    screen_h = self.winfo_screenheight()
    x = (screen_w - width) // 2
    y = (screen_h - height) // 2
    self.geometry(f"{width}x{height}+{x}+{y}")

def _build_ui(self):
    """Будує інтерфейс форми."""

    # Заголовок
    ctk.CTkLabel(
        self,
        text="Параметри проєкту",
        font=ctk.CTkFont(size=20, weight="bold")
    ).pack(pady=(20, 5))

    ctk.CTkLabel(
        self,
        text="Заповніть всі поля для отримання оцінки  
ризик",
        font=ctk.CTkFont(size=12),
        text_color="gray"
    ).pack(pady=(0, 15))

```

```

# Прокручуваний фрейм для полів форми
# ScrollableFrame дозволяє прокручувати якщо полів
забагато
scroll_frame = ctk.CTkScrollableFrame(self, width=500,
height=480)
scroll_frame.pack(padx=20, pady=(0, 10), fill="both",
expand=True)

# Генеруємо поля для кожного параметра з FEATURES
for feature in FEATURES:
    self._create_field(scroll_frame, feature)

# Кнопки внизу форми
btn_frame = ctk.CTkFrame(self, fg_color="transparent")
btn_frame.pack(pady=10)

ctk.CTkButton(
    btn_frame,
    text="🔍 Оцінити ризик",
    font=ctk.CTkFont(size=14, weight="bold"),
    width=200,
    height=45,
    command=self._submit
).pack(side="left", padx=10)

ctk.CTkButton(
    btn_frame,
    text="🧼 Очистити",
    font=ctk.CTkFont(size=13),
    width=130,
    height=45,
    fg_color="gray",
    hover_color="#555555",
    command=self._clear_form
).pack(side="left", padx=10)

def _create_field(self, parent, feature: dict):
    """
    Створює одне поле форми для параметра.
    Кожне поле складається з:

```

- Label з назвою параметра
- Entry для введення значення
- Label для відображення помилки валідації

```
key = feature["key"]
```

```
# Контейнер для одного поля
```

```
field_frame = ctk.CTkFrame(parent,  
fg_color="transparent")  
field_frame.pack(fill="x", pady=6)
```

```
# Назва параметра
```

```
ctk.CTkLabel(  
    field_frame,  
    text=feature["label"],  
    font=ctk.CTkFont(size=13, weight="bold"),  
    anchor="w"  
).pack(fill="x")
```

```
# Підказка під назвою
```

```
ctk.CTkLabel(  
    field_frame,  
    text=feature["hint"],  
    font=ctk.CTkFont(size=11),  
    text_color="gray",  
    anchor="w"  
).pack(fill="x")
```

```
# Поле введення з дефолтним значенням
```

```
entry = ctk.CTkEntry(  
    field_frame,  
    placeholder_text=str(feature["default"]),  
    width=460,  
    height=36  
)
```

```
entry.insert(0, str(feature["default"])) # вставляємо  
дефолтне значення
```

```
entry.pack(fill="x", pady=(3, 0))
```

```
# Мітка для помилки (спочатку порожня)
```

```

error_label = ctk.CTkLabel(
    field_frame,
    text="",
    font=ctk.CTkFont(size=11),
    text_color="#e74c3c", # червоний колір для помилок
    anchor="w"
)
error_label.pack(fill="x")

# Зберігаємо посилання на поля для подальшого доступу
self.entries[key] = entry
self.error_labels[key] = error_label

def _validate_all(self) -> tuple[bool, dict]:
    """
    Валідуює всі поля форми.

    Повертає (all_valid, converted_values):
    - all_valid: True якщо всі поля валідні
    - converted_values: словник з конвертованими значеннями
    """
    all_valid = True
    values = {}

    for feature in FEATURES:
        key = feature["key"]
        raw_value = self.entries[key].get() # отримуємо
текст з поля

        is_valid, error_msg, converted =
validate_input(key, raw_value)

        if is_valid:
            # Очищаємо помилку якщо поле валідне
            self.error_labels[key].configure(text="")
            values[key] = converted
        else:
            # Показуємо помилку під полем
            self.error_labels[key].configure(text=f"⚠️
{error_msg}")

```

```

        all_valid = False

    return all_valid, values

def _submit(self):
    """Обробляє натискання кнопки 'Оцінити ризик'."""
    logger.info("Спроба відправки форми")

    # Валідуємо всі поля
    all_valid, values = self._validate_all()

    if not all_valid:
        # Показуємо загальне повідомлення про помилки
        mb.showwarning(
            "Помилка валідації",
            "Будь ласка, виправте помилки у виділених
полях."
        )
        logger.warning("Форма не пройшла валідацію")
        return

    # Всі поля валідні – робимо передбачення
    logger.info(f"Параметри форми: {values}")

    result = predict(self.model, values)
    logger.info(f"Результат оцінки:
{result['risk_label']}")

    # Закриваємо форму і відкриваємо вікно результатів
    self.destroy()

    from app.gui.result_view import ResultView
    ResultView(self.master, values, result)

def _clear_form(self):
    """Очищає всі поля і повертає дефолтні значення."""
    for feature in FEATURES:
        key = feature["key"]
        self.entries[key].delete(0, "end")

```

```

        self.entries[key].insert(0,
str(feature["default"]))
        self.error_labels[key].configure(text="")

logger.info("Форму очищено")

```

app/gui/result_view.py — Вікно результатів

Відображає результати оцінки та вбудовує matplotlib графіки безпосередньо у вікно tkinter.

```

# app/gui/result_view.py
# Вікно відображення результатів оцінки ризику з графіками

import customtkinter as ctk
import tkinter.messagebox as mb
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

from app.core.logger import logger
from app.ml.features import FEATURES, RISK_COLORS
from app.ml.report import generate_report

class ResultView(ctk.CTkToplevel):
    """
    Вікно результатів оцінки ризику.
    Показує рівень ризику, графіки та кнопку збереження звіту.
    """

    def __init__(self, parent, input_data: dict, result: dict):
        super().__init__(parent)

        self.input_data = input_data
        self.result = result

        self.title("Результат оцінки ризику")
        self.geometry("700x750")
        self.resizable(False, True)

```

```

self._center_window(700, 750)

self.grab_set()
self.focus_set()

logger.info("Вікно результатів відкрито")
self._build_ui()

def _center_window(self, width: int, height: int):
    """Центрує вікно на екрані."""
    screen_w = self.winfo_screenwidth()
    screen_h = self.winfo_screenheight()
    x = (screen_w - width) // 2
    y = (screen_h - height) // 2
    self.geometry(f"{width}x{height}+{x}+{y}")

def _build_ui(self):
    """Будує інтерфейс вікна результатів."""

    risk_level = self.result["risk_level"]
    risk_label = self.result["risk_label"]
    color = RISK_COLORS[risk_level]

    # --- Заголовок з рівнем ризику ---
    header_frame = ctk.CTkFrame(self,
fg_color="transparent")
    header_frame.pack(pady=(20, 5))

    ctk.CTkLabel(
        header_frame,
        text="Результат оцінки",
        font=ctk.CTkFont(size=20, weight="bold")
    ).pack()

    # Велика кольорова мітка рівня ризику
    risk_icons = {0: "🟢", 1: "🟡", 2: "🔴"}
    ctk.CTkLabel(
        header_frame,
        text=f"{risk_icons[risk_level]} Рівень ризику: {risk_label}",

```

```

        font=ctk.CTkFont(size=22, weight="bold"),
        text_color=color
    ).pack(pady=10)

    # Ймовірності у вигляді тексту
    probs = self.result["probabilities"]
    prob_text = " | ".join(
        [f"{label}: {prob*100:.1f}%" for label, prob in
probs.items()]
    )
    ctk.CTkLabel(
        header_frame,
        text=prob_text,
        font=ctk.CTkFont(size=12),
        text_color="gray"
    ).pack()

    # --- Графіки ---
    self._build_charts()

    # --- Кнопки ---
    btn_frame = ctk.CTkFrame(self, fg_color="transparent")
    btn_frame.pack(pady=15)

    ctk.CTkButton(
        btn_frame,
        text="💾 Зберегти звіт (Excel)",
        font=ctk.CTkFont(size=13, weight="bold"),
        width=220,
        height=42,
        command=self._save_report
    ).pack(side="left", padx=10)

    ctk.CTkButton(
        btn_frame,
        text="🔍 Нова оцінка",
        font=ctk.CTkFont(size=13),
        width=160,
        height=42,
        fg_color="gray",

```



```

        hover_color="#555555",
        command=self._new_assessment
    ).pack(side="left", padx=10)

    ctk.CTkButton(
        btn_frame,
        text="X Закрити",
        font=ctk.CTkFont(size=13),
        width=120,
        height=42,
        fg_color="gray",
        hover_color="#555555",
        command=self.destroy
    ).pack(side="left", padx=10)

def _build_charts(self):
    """Будує два графіки: pie chart і bar chart важливості
    факторів."""

    # Визначаємо чи темна тема активна
    is_dark = ctk.get_appearance_mode().lower() == "dark"
    bg_color = "#2b2b2b" if is_dark else "#f0f0f0"
    text_color = "white" if is_dark else "black"

    # Створюємо фігуру з двома підграфіками
    fig, (ax1, ax2) = plt.subplots(
        1, 2,
        figsize=(6.5, 3.2),
        facecolor=bg_color
    )

    # --- Графік 1: Pie chart ймовірностей ---
    probs = self.result["probabilities"]
    labels = list(probs.keys())
    sizes = list(probs.values())
    pie_colors = [
        RISK_COLORS[0], # зелений для Низького
        RISK_COLORS[1], # помаранчевий для Середнього
        RISK_COLORS[2]  # червоний для Високого
    ]

```

```

# Виділяємо сектор з найвищим ризиком
explode = [0.05] * len(sizes)

wedges, texts, autotexts = ax1.pie(
    sizes,
    labels=labels,
    colors=pie_colors,
    explode=explode,
    autopct="%1.1f%%",
    startangle=90,
    textprops={"color": text_color, "fontsize": 9}
)

for autotext in autotexts:
    autotext.set_color(text_color)

ax1.set_title(
    "Розподіл ймовірностей",
    color=text_color,
    fontsize=11,
    pad=10
)
ax1.set_facecolor(bg_color)

# --- Графік 2: Bar chart важливості факторів ---
importance = self.result["feature_importance"]

# Беремо топ-6 найважливіших факторів
top_items = list(importance.items())[:6]
factor_keys = [item[0] for item in top_items]
factor_values = [item[1] * 100 for item in top_items]

# Отримуємо короткі назви факторів для підписів
feature_short_names = {f["key"]:
f["label"].split("(")[0].strip()
                        for f in FEATURES}
factor_labels = [feature_short_names.get(k, k) for k in
factor_keys]

```

```

# Скорочуємо довгі назви для читабельності
factor_labels = [
    label[:20] + "..." if len(label) > 20 else label
    for label in factor_labels
]

bar_colors = ["#3498db"] * len(factor_values)
bars = ax2.barh(
    factor_labels,
    factor_values,
    color=bar_colors,
    edgecolor="none"
)

# Підписи значень на барах
for bar, val in zip(bars, factor_values):
    ax2.text(
        bar.get_width() + 0.3,
        bar.get_y() + bar.get_height() / 2,
        f"{val:.1f}%",
        va="center",
        color=text_color,
        fontsize=8
    )

ax2.set_title(
    "Вплив факторів (топ-6)",
    color=text_color,
    fontsize=11,
    pad=10
)
ax2.set_facecolor(bg_color)
ax2.tick_params(colors=text_color, labelsize=8)
ax2.spines["top"].set_visible(False)
ax2.spines["right"].set_visible(False)
ax2.spines["bottom"].set_color("gray")
ax2.spines["left"].set_color("gray")
ax2.set_xlabel("Важливість (%)", color=text_color,
    fontsize=9)
ax2.invert_yaxis() # найважливіший фактор зверху

```

```

plt.tight_layout(pad=1.5)

# Вбудовуємо matplotlib графік у tkinter вікно
canvas = FigureCanvasTkAgg(fig, master=self)
canvas.draw()
canvas.get_tk_widget().pack(padx=15, pady=5,
fill="both")

# Закриваємо figure щоб не витікала пам'ять
plt.close(fig)

def _save_report(self):
    """Зберігає Excel звіт і показує повідомлення про
успіх."""
    try:
        filepath = generate_report(self.input_data,
self.result)
        logger.info(f"Звіт збережено користувачем:
{filepath}")
        mb.showinfo(
            "Звіт збережено",
            f"Звіт успішно збережено:\n{filepath}"
        )
    except Exception as e:
        logger.error(f"Помилка збереження звіту: {e}")
        mb.showerror(
            "Помилка",
            f"Не вдалося зберегти звіт:\n{str(e)}"
        )

def _new_assessment(self):
    """Закриває поточне вікно і відкриває нову форму
оцінки."""
    self.destroy()
    from app.gui.input_form import InputForm
    InputForm(self.master)
    logger.info("Розпочато нову оцінку")

```

app/gui/settings_view.py — Налаштування

Дозволяє змінювати тему та параметри моделі з негайним застосуванням і збереженням у config.json.

```
# app/gui/settings_view.py
# Вікно налаштувань програми

import customtkinter as ctk
import tkinter.messagebox as mb

from app.core.config import load_config, save_config, get
from app.core.logger import logger

class SettingsView(ctk.CTkToplevel):
    """Вікно налаштувань – дозволяє змінювати тему та параметри моделі."""

    def __init__(self, parent):
        super().__init__(parent)

        self.parent = parent
        self.title("Налаштування")
        self.geometry("450x600")
        self.resizable(False, False)
        self._center_window(450, 500)

        self.grab_set()
        self.focus_set()

        # Завантажуємо поточний конфіг
        self.config = load_config()

        logger.info("Вікно налаштувань відкрито")
        self._build_ui()

    def _center_window(self, width: int, height: int):
        screen_w = self.winfo_screenwidth()
        screen_h = self.winfo_screenheight()
        x = (screen_w - width) // 2
```

```
y = (screen_h - height) // 2
self.geometry(f"{width}x{height}+{x}+{y}")

def _build_ui(self):
    """Будує інтерфейс вікна налаштувань."""

    ctk.CTkLabel(
        self,
        text="Налаштування",
        font=ctk.CTkFont(size=20, weight="bold")
    ).pack(pady=(20, 15))

    # --- Секція: Зовнішній вигляд ---
    self._section_label("☺ Зовнішній вигляд")

    # Вибір теми
    theme_frame = ctk.CTkFrame(self,
fg_color="transparent")
    theme_frame.pack(fill="x", padx=30, pady=5)

    ctk.CTkLabel(
        theme_frame,
        text="Тема інтерфейсу:",
        font=ctk.CTkFont(size=13)
    ).pack(side="left")

    self.theme_var = ctk.StringVar(
        value=self.config.get("appearance",
    {}).get("theme", "dark")
    )

    ctk.CTkOptionMenu(
        theme_frame,
        values=["dark", "light", "system"],
        variable=self.theme_var,
        width=140
    ).pack(side="right")

    # Вибір кольорової схеми
```

```

        color_frame = ctk.CTkFrame(self,
fg_color="transparent")
        color_frame.pack(fill="x", padx=30, pady=5)

        ctk.CTkLabel(
            color_frame,
            text="Кольорова схема:",
            font=ctk.CTkFont(size=13)
        ).pack(side="left")

        self.color_var = ctk.StringVar(
            value=self.config.get("appearance",
{ }).get("color_scheme", "blue")
        )

        ctk.CTkOptionMenu(
            color_frame,
            values=["blue", "green", "dark-blue"],
            variable=self.color_var,
            width=140
        ).pack(side="right")

        # --- Секція: Параметри моделі ---
        self._section_label("🤖 Параметри моделі")

        # Кількість дерев
        trees_frame = ctk.CTkFrame(self,
fg_color="transparent")
        trees_frame.pack(fill="x", padx=30, pady=5)

        ctk.CTkLabel(
            trees_frame,
            text="Кількість дерев (50-500):",
            font=ctk.CTkFont(size=13)
        ).pack(side="left")

        self.trees_entry = ctk.CTkEntry(trees_frame, width=80)
        self.trees_entry.insert(
            0, str(self.config.get("model",
{ }).get("n_estimators", 100))

```

```

)
self.trees_entry.pack(side="right")

# Розмір тестової вибірки
test_frame = ctk.CTkFrame(self, fg_color="transparent")
test_frame.pack(fill="x", padx=30, pady=5)

ctk.CTkLabel(
    test_frame,
    text="Розмір тесту (0.1-0.4):",
    font=ctk.CTkFont(size=13)
).pack(side="left")

self.test_entry = ctk.CTkEntry(test_frame, width=80)
self.test_entry.insert(
    0, str(self.config.get("model",
{}}).get("test_size", 0.2))
)
self.test_entry.pack(side="right")

# --- Версія програми (тільки читання) ---
self._section_label("i Інформація")

info_frame = ctk.CTkFrame(self, fg_color="transparent")
info_frame.pack(fill="x", padx=30, pady=5)

version = get("app.version", "1.0.0")
ctk.CTkLabel(
    info_frame,
    text=f"Версія програми: {version}",
    font=ctk.CTkFont(size=13),
    text_color="gray"
).pack(side="left")

# --- Кнопки ---
btn_frame = ctk.CTkFrame(self, fg_color="transparent")
btn_frame.pack(pady=20)

ctk.CTkButton(
    btn_frame,

```



```

        text="💾 Зберегти",
        font=ctk.CTkFont(size=13, weight="bold"),
        width=150,
        height=40,
        command=self._save
    ).pack(side="left", padx=10)

    ctk.CTkButton(
        btn_frame,
        text="✖ Скасувати",
        font=ctk.CTkFont(size=13),
        width=130,
        height=40,
        fg_color="gray",
        hover_color="#555555",
        command=self.destroy
    ).pack(side="left", padx=10)

def _section_label(self, text: str):
    """Створює заголовок секції з роздільником."""
    ctk.CTkLabel(
        self,
        text=text,
        font=ctk.CTkFont(size=13, weight="bold"),
        anchor="w"
    ).pack(fill="x", padx=30, pady=(15, 2))

    ctk.CTkFrame(self, height=1, fg_color="gray").pack(
        fill="x", padx=30, pady=(0, 5)
    )

def _save(self):
    """Валідує і зберігає налаштування в config.json."""

    # Валідація кількості дерев
    try:
        n_estimators = int(self.trees_entry.get())
        if not (50 <= n_estimators <= 500):
            raise ValueError
    except ValueError:

```

```

        mb.showwarning(
            "Помилка",
            "Кількість дерев має бути цілим числом від 50
до 500"
        )
        return

    # Валідація розміру тестової вибірки
    try:
        test_size =
float(self.test_entry.get().replace(",", "."))
        if not (0.1 <= test_size <= 0.4):
            raise ValueError
    except ValueError:
        mb.showwarning(
            "Помилка",
            "Розмір тесту має бути числом від 0.1 до 0.4"
        )
        return

    # Зберігаємо в конфіг
    self.config["appearance"]["theme"] =
self.theme_var.get()
    self.config["appearance"]["color_scheme"] =
self.color_var.get()
    self.config["model"]["n_estimators"] = n_estimators
    self.config["model"]["test_size"] = test_size

    save_config(self.config)
    logger.info(f"Налаштування збережено:
тема={self.theme_var.get()}")

    # Застосовуємо тему одразу
    ctk.set_appearance_mode(self.theme_var.get())
    ctk.set_default_color_theme(self.color_var.get())

    mb.showinfo("Збережено", "Налаштування збережено
успішно!")
    self.destroy()

```

ВИСНОВОК ДО ЧАСТИНИ 3

У ході виконання третьої частини лабораторної роботи було сформовано повний пакет супровідної документації для програмного забезпечення **Project Risk AI v1.0.0**.

Розроблена інструкція користувача охоплює всі етапи взаємодії з програмою: від встановлення та налаштування середовища до детального опису кожного елемента інтерфейсу та практичних прикладів використання. Документ містить таблицю можливих помилок та способів їх усунення, що забезпечує зручність використання системи для недосвідченого користувача.

Схеми структур даних наочно відображають організацію файлів проєкту та зв'язки між модулями. Модульна архітектура програми, де кожен компонент виконує чітко визначену функцію, забезпечує зручність супроводу та розширення системи.

Блок-схеми алгоритмів розроблені відповідно до стандарту ДСТУ ISO 5807:2016 і охоплюють три ключові функції: валідацію вхідних даних, передбачення ризику моделлю машинного навчання та генерацію звіту.

Лістинг програми містить повний код усіх десяти модулів з коментарями до кожного блоку коду.